

+ 23

Tracy:

A Profiler You Don't Want to Miss

MARCOS SLOMP



20
23



October 01 - 06

Tracy: a profiler you don't want to miss

Marcos Slomp

Senior Research Engineer II



Why Tracy?

Real-time workflow

(analyze application **while it is running**)

Precise measurements

(**nanosecond** resolution)

Negligible overhead

(a few **nanoseconds** per zone)

Cross-platform

(Windows, Linux, macOS, iOS, Android, WASM*)

Hybrid profiling capabilities

(**sampling** and/or **instrumentation**)

(**CPU** and **GPU** instrumentation)

Tracing capabilities

(values, messages, plots, allocations, ...)

Hassle-free integration

(single source* file, and header*)

Free and Open source



PAY WHAT YOU WANT

\$000

Exc. Internet fees
Inc. source code

1-555-PROFILE

FREE Trial! Instant Delivery!
Lifetime Money Back Guarantee (less S&H).

<https://github.com/wolfpld/tracy>

Why Tracy?

Real-time workflow
(analyze application **while it is running**)

Precise measurements
(**nanosecond** resolution)

Negligible overhead
(a few **nanoseconds** per zone)

Cross-platform
(Windows, Linux, macOS, iOS, Android, WASM*)

Hybrid profiling capabilities
(**sampling** and/or **instrumentation**)

Tracing capabilities
(values, messages, plots, allocations, ...)

Free and Open source



Kudos to Bartosz Taudul
creator and maintainer of Tracy



PAY WHAT YOU WANT

\$000

Exc. Internet fees
Inc. source code

1-555-PROFILE

FREE Trial! Instant Delivery!

Lifetime Money Back Guarantee (less S&H).

<https://github.com/wolfpld/tracy>

Why Tracy?

Real-time workflow
(analyze application **while it is running**)

Precise measurements
(**nanosecond** resolution)

Negligible overhead
(a few **nanoseconds** per zone)

Cross-platform
(Windows, Linux, macOS, iOS, Android, WASM*)

Hybrid profiling capabilities
(**sampling** and/or **instrumentation**)

Tracing capabilities
(values, messages, plots, allocations, ...)

Free and Open source



Kudos to David Farrell **Md**
for demonstrating Tracy's potential to me!



PAY WHAT YOU WANT

\$000

Exc. Internet fees
Inc. source code

1-555-PROFILE

FREE Trial! Instant Delivery!

Lifetime Money Back Guarantee (less S&H).

<https://github.com/wolfpld/tracy>

DISCLAIMER: this talk is **NOT** about “Tracy vs. other tools”

Telemetry	http://www.radgametools.com/telemetry.htm
Microprofile	https://github.com/jonasmr/microprofile
Microprofile 2	https://github.com/zeux/microprofile
Optick (Brofiler)	https://github.com/bombomby/optick
Palanteer	https://github.com/dfeneyrou/palanteer
Orbit Profiler	https://github.com/google/orbit
SuperLuminal	https://superluminal.eu
Perfetto	https://perfetto.dev
MicroProfiler	https://github.com/tyoma/micro-profiler
EasyProfiler	https://github.com/yse/easy_profiler
Oprofile	https://oprofile.sourceforge.io/examples/
Coz	https://github.com/plasma-umass/coz
VerySleepy	https://github.com/VerySleepy/verysleepy
LukeStackwalker	http://lukestackwalker.sourceforge.net
dprofiler	https://github.com/xwlan/dprofiler
hotspot	https://github.com/KDAB/hotspot
heaptrack	https://github.com/KDE/heaptrack
spall	https://handmade.network/p/333/spall/
geiger	https://github.com/david-grs/geiger
Xpedit	https://github.com/morganstanley/Xpedit
Intel IACA	https://www.intel.com/content/www/us/en/developer/articles/tool/architecture-code-analyzer.html

The Obligatory Quote

premature optimization is the root of all evil.

Donald E. Knuth

The Obligatory Quote, in proper context

“There is no doubt that the grail of efficiency leads to abuse. Programmers **waste enormous amounts of time** thinking about, or **worrying** about, the **speed of noncritical parts** of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered. We *should* **forget about small efficiencies**, say about 97% of the time: **premature optimization is the root of all evil**. Yet we **should not pass up our opportunities in that critical 3%.**”

Donald E. Knuth

*Structured Programming with go to Statements.
ACM Computing Surveys 6, 4 (Dec. 1974), 261–301.*

**Mathieu Ropert**

<https://www.youtube.com/watch?v=dToaeplXW4s>

*“The real problem is that
programmers have spent far too
much time worrying about
efficiency in the wrong places and
at the wrong times”*

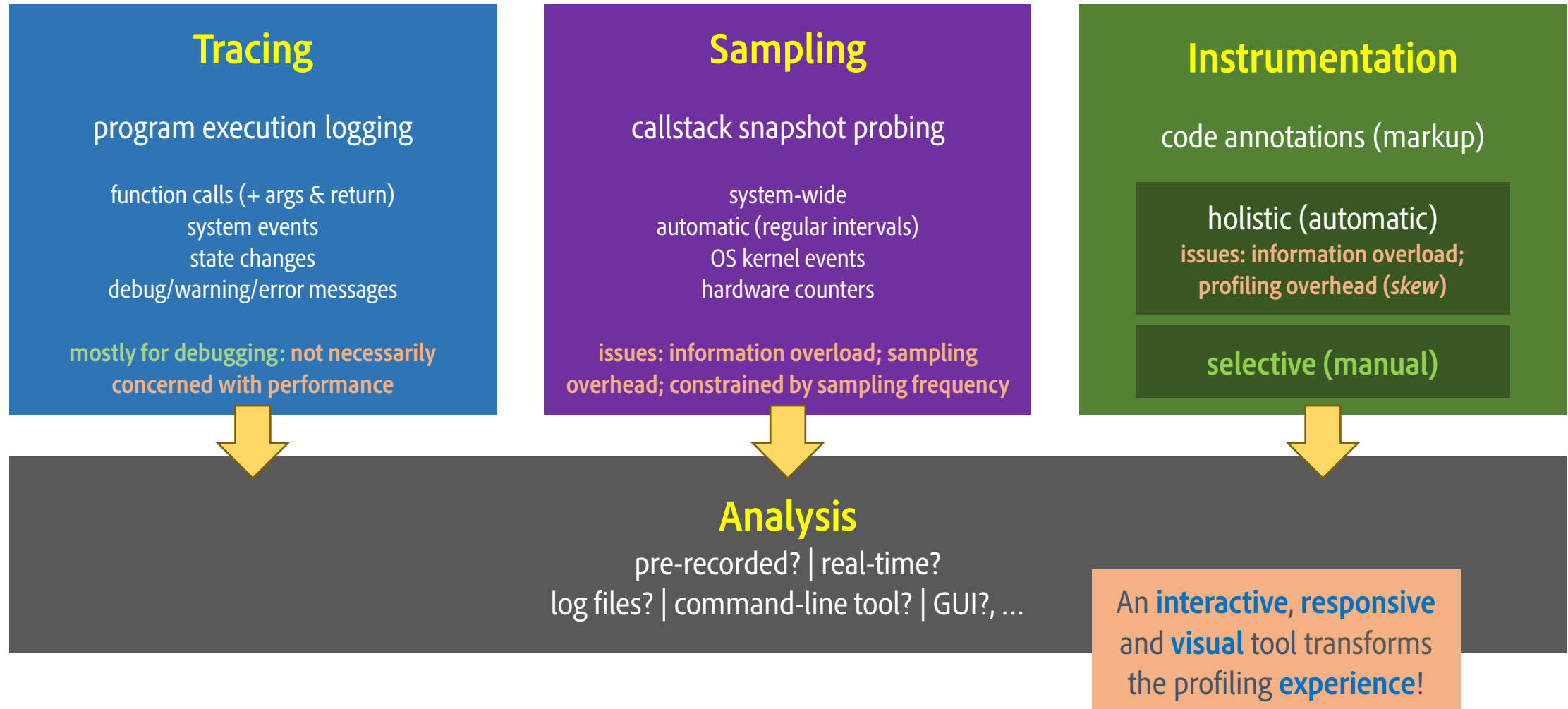


“C++ programmers talk all the time about **efficiency**,
but they generally don’t know how to **measure**.”

- Bjarne Stroustrup

CppCon 2023 Opening Keynote
(literally three days ago!)

Closing the Profiling Zoo



Closing the Profiling Zoo

Tracing

program execution logging

- function calls (+ args & return)
- system events
- state changes
- debug/warning/error messages

mostly for debugging: not necessarily
concerned with performance



Tracy can do it all!

Instrumentation

code annotations (markup)

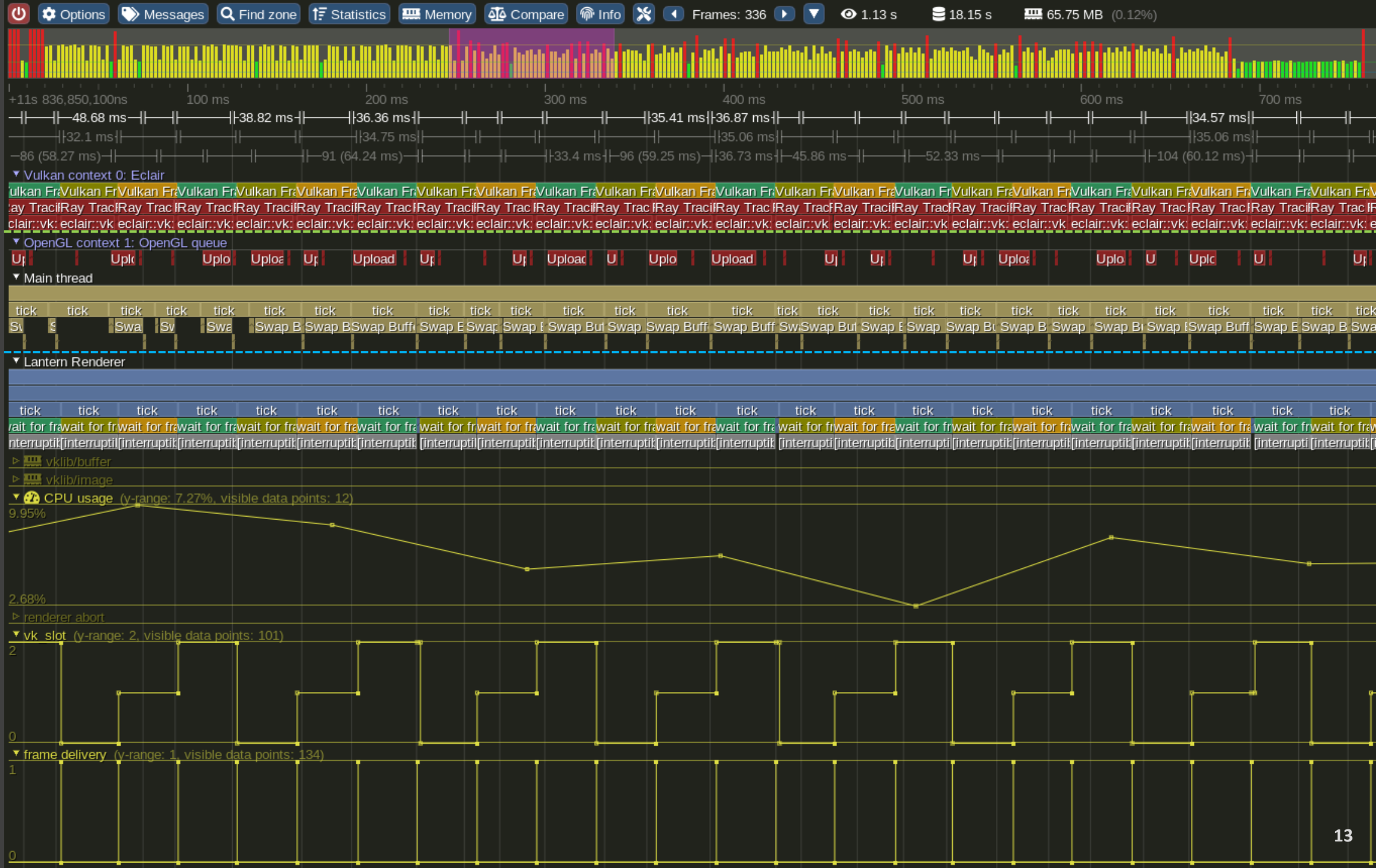
holistic (automatic)
issues: information overload;
profiling overhead (*skew*)

selective (manual)



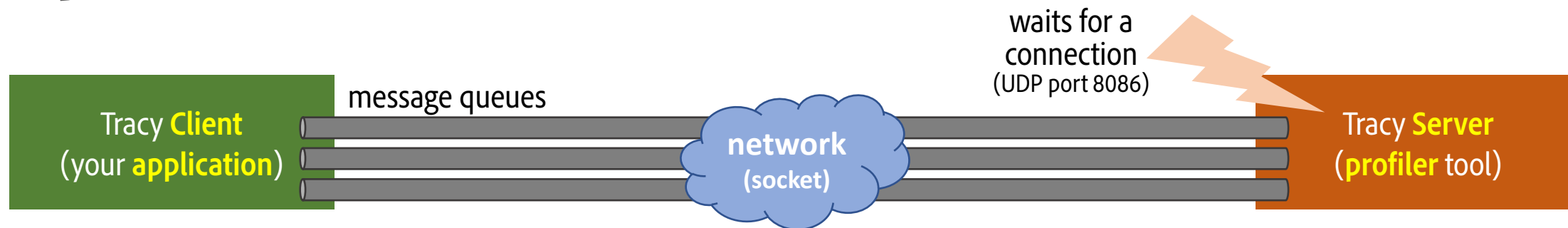
interactive, responsive
visual tool transforms
profiling experience!

Tracy Profiler GUI





Tracy Architecture



Basic integration (the *responsible* way):

1. simply `#include <Tracy.hpp>` whenever needed
2. compile `TracyClient.cpp` alongside your application
3. enable Tracy by defining `TRACY_ENABLE` in your build

Quick integration (the *YOLO* way):

```
#define TRACY_ENABLE
#include <Tracy.hpp>
```

anywhere where needed

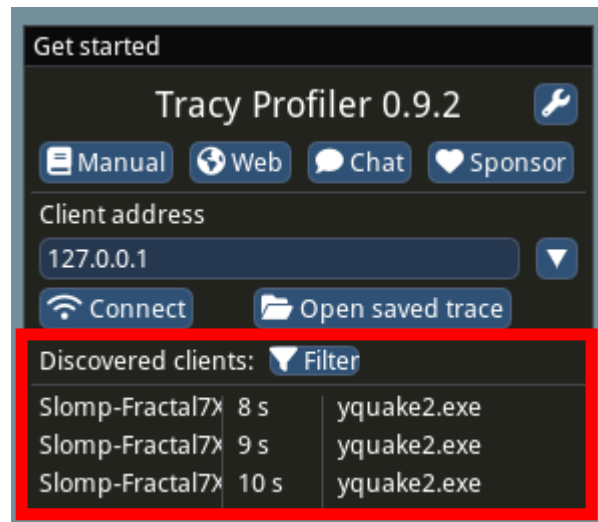
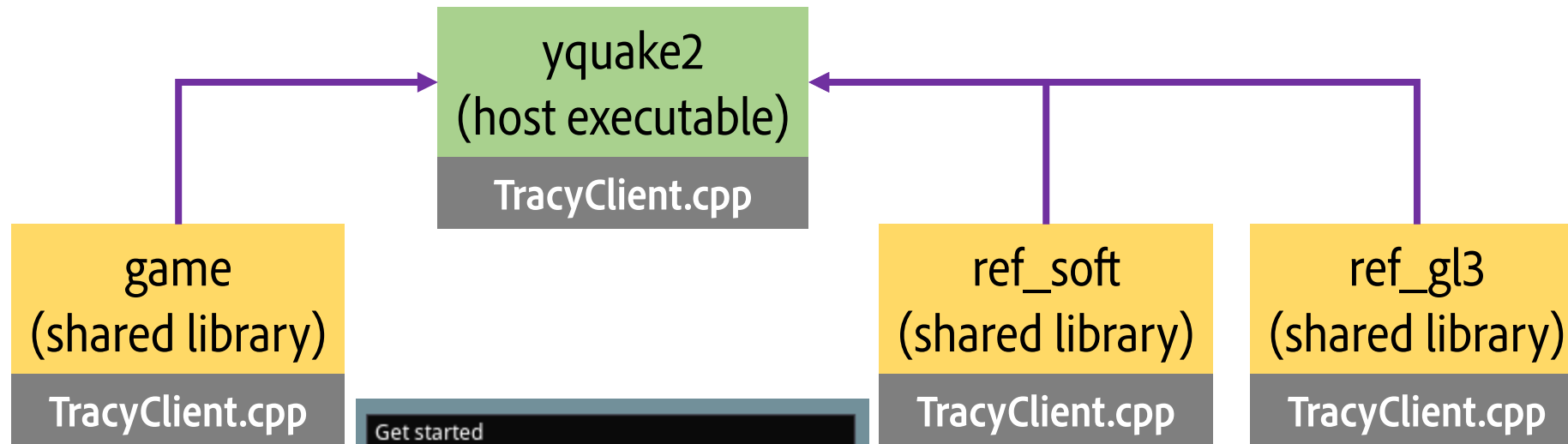
```
#define TRACY_ENABLE
#include <TracyClient.cpp>
```

in *one* of the source files of your project

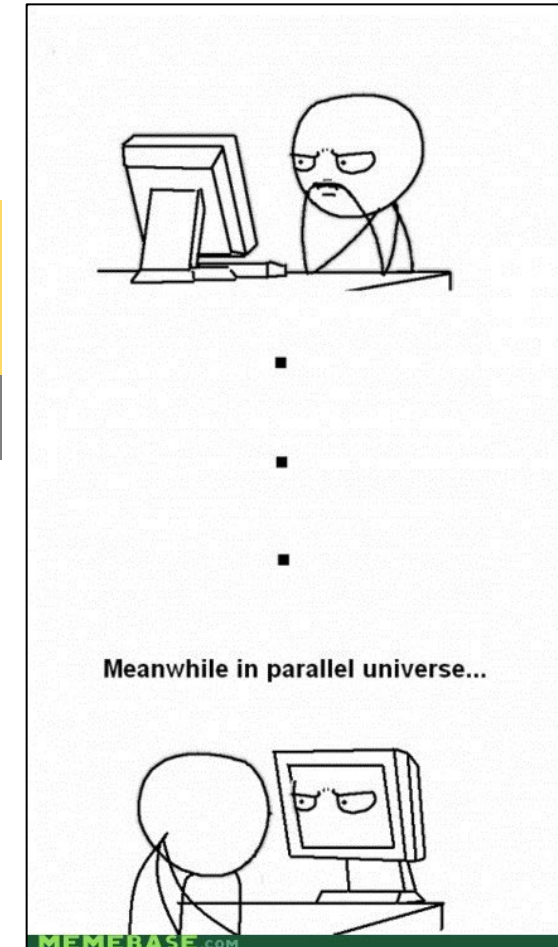


download pre-built binary (**Windows-only**):
<https://github.com/wolfpld/tracy/releases>
or **build it yourself!**

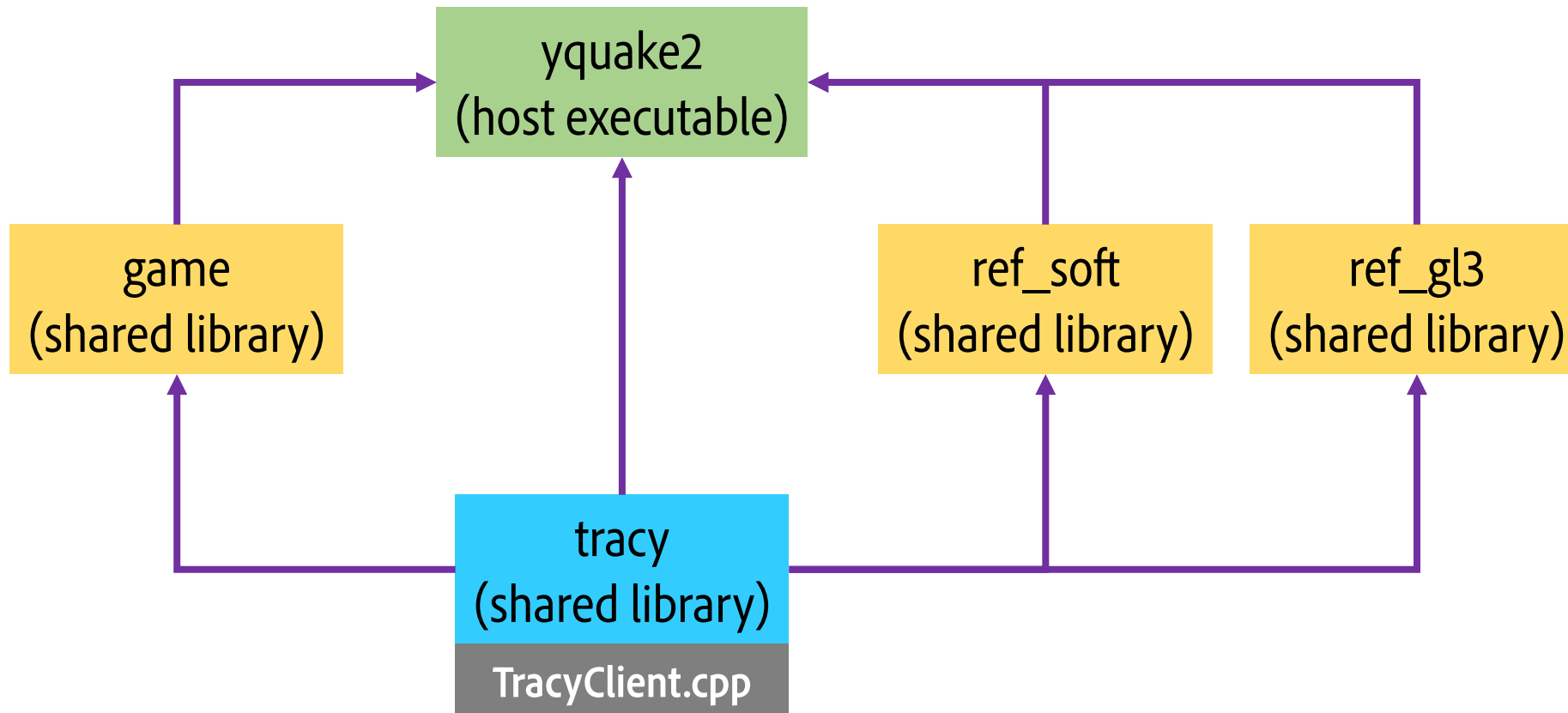
Tracy integration across shared libraries



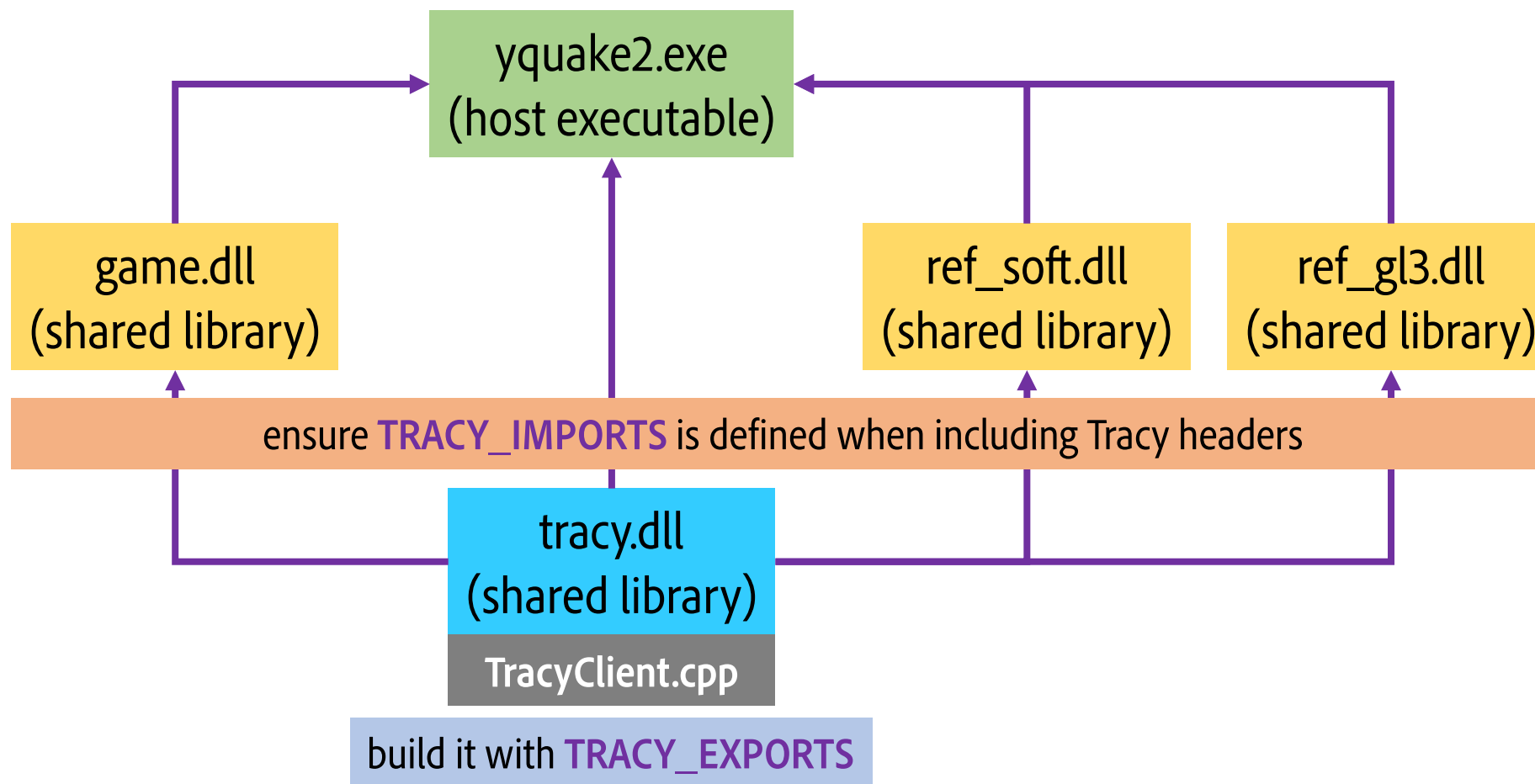
Behold the Tracy Multiverse!
(Tracy Manual, Section 2.1.5)



Tracy integration across shared libraries



Tracy integration across shared libraries (**Windows**)



Building the Tracy Profiler from source



1. Obtain the **dependencies** (**capstone**, **glfw**, **freetype**):
`<tracy-repo>\vcpkg\install_vcpkg_dependencies.bat`

2. Build with the **Visual Studio** solution:
`<tracy-repo>\profiler\build\win32\Tracy.sln`

Tracy.exe



1. Obtain the **dependencies** (**capstone**, **freetype**) (**glfw** or **libxkbcommon**, **wayland**, **libglvnd**, and maybe **libegl** too):
(use whatever package manager your distro provides)

2. Build with **make**:
`<tracy-repo>/profiler/build/unix $ make -j[N]`

Tracy-release



1. Obtain the **dependencies** (**capstone**, **glfw**, **freetype**):
`$ brew install capstone glfw freetype`

2. Build with **make**:
`<tracy-repo>/profiler/build/unix $ make -j[N]`

Tracy-release

Zoning...

```
{ this is a zone scope (RAII) }
```

```
int foo()  
{
```

```
    ...  
    ...  
    if (!spoon) {  
        ...  
        ...  
    }  
    ...  
    {  
        ...  
        ...  
    }  
}
```



Source: <https://www.mobygames.com/game/657/simcity-2000/promo/group-17786/image-185051/>

Tracy Zoning 101

```
int foo()  
{
```

```
    ZoneScoped;  
    ...
```

```
    ...
```

```
    if (!spoon) {
```

```
        ZoneScoped;  
        ...
```

```
    }  
    ...
```

```
    ...
```

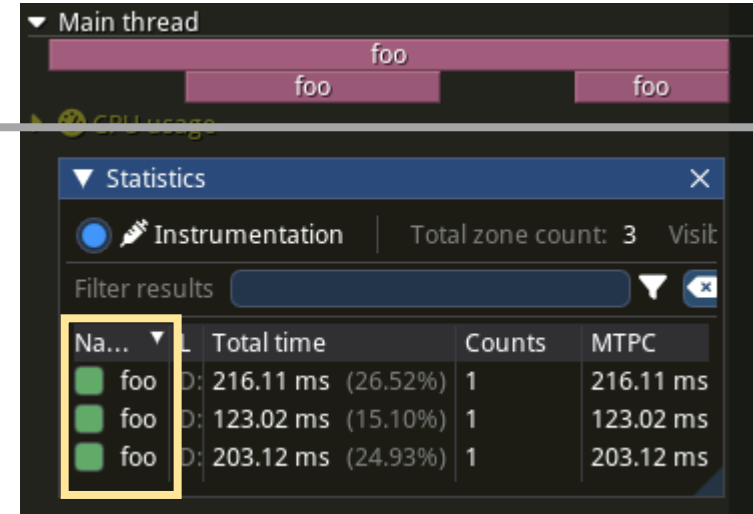
```
    {
```

```
        ZoneScoped;  
        ...
```

```
    }  
}
```

thread **timeline**

zone **statistics**



- **ZoneScoped**

- the “**bread-and-butter**” of Tracy
- only **1 zone per scope**
- zone **name is inferred** (enclosing function name)
- zone **color is automatic** (per-thread and depth)

Tracy Zoning 101

```
int foo()  
{
```

```
    ZoneScoped;
```

```
    ...
```

```
    if (!spoon) {
```

```
        ZoneScopedN("matrix");
```

```
        ...
```

```
    }
```

```
    ...
```

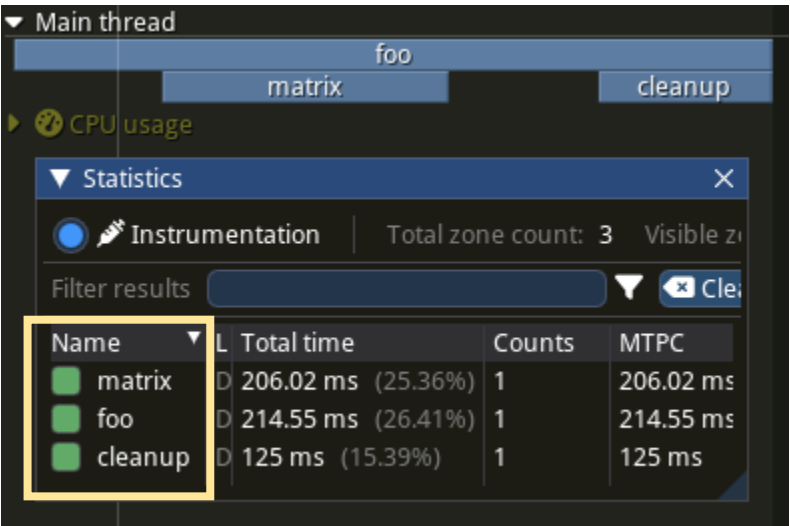
```
    {
```

```
        ZoneScopedN("cleanup");
```

```
        ...
```

```
    }
```

```
}
```



Name	Total time	Counts	MTPC
matrix	206.02 ms (25.36%)	1	206.02 ms
foo	214.55 ms (26.41%)	1	214.55 ms
cleanup	125 ms (15.39%)	1	125 ms

- **ZoneScopedN**

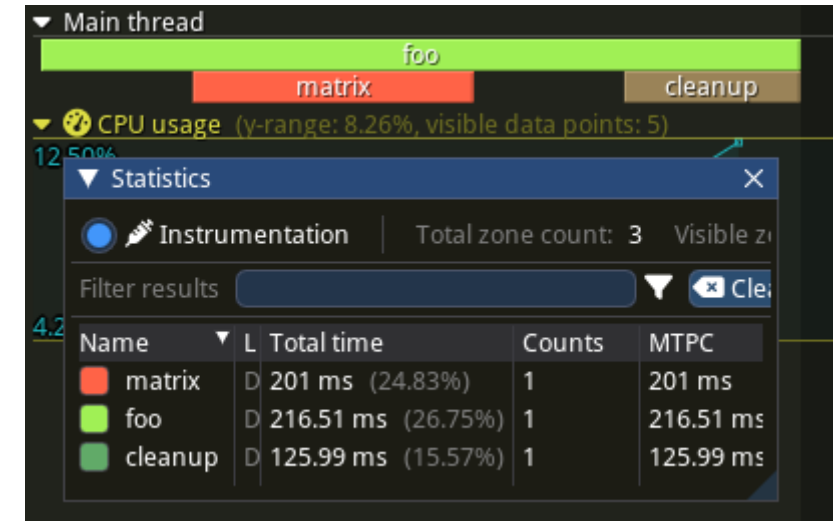
- only 1 zone per scope
- zone color is automatic (per-thread and depth)
- **you name it!** (must be *persistent string literal*)

Tracy manual, Section 3.1

the provided string data must be accessible at any time in program execution (*this also includes the time after exiting the main function*). The string also cannot be changed. This basically means that the only option is to use a string literal

Tracy Zoning 101

```
int foo()  
{  
    ZoneScopedC(0xA0F055);  
    ...  
    if (!spoon) {  
        ZoneScopedNC("matrix", tracy::Color::Tomato);  
        ...  
    }  
    ...  
    {  
        ZoneScopedN("cleanup");  
        ...  
    }  
}
```



- **ZoneScoped[N]C**
 - only 1 zone per scope
 - you [may] name it
 - **you pick the color**
 - hexadecimal RGB triplet: 0xRRGGBB
 - or [X11 color names](#) (through `tracy::Color`)

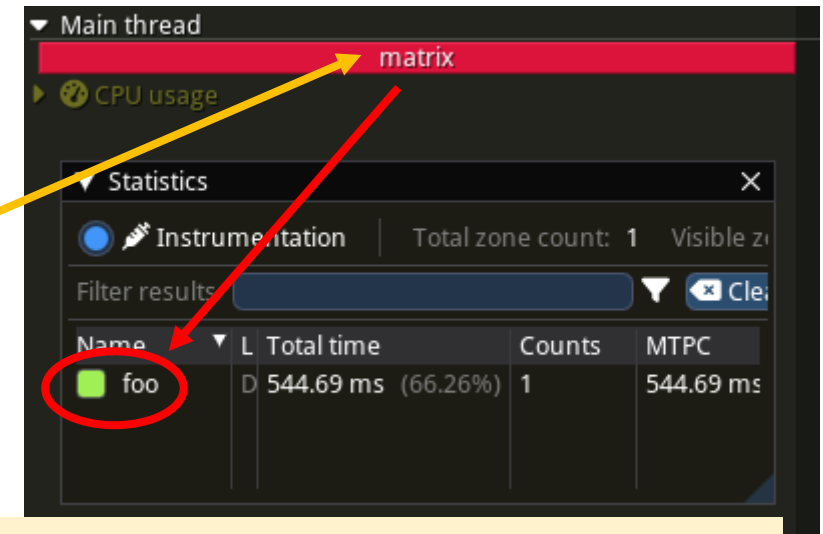


Tracy Zoning 201

```
int foo()  
{  
    ZoneScoped;  
    ...  
    if (!spoon) {  
        ZoneName("matrix", 6);  
        ...  
    }  
    ...  
    if (error != 0) {  
        ZoneColor(Crimson);  
        ...  
    }  
}
```

strlen("matrix")

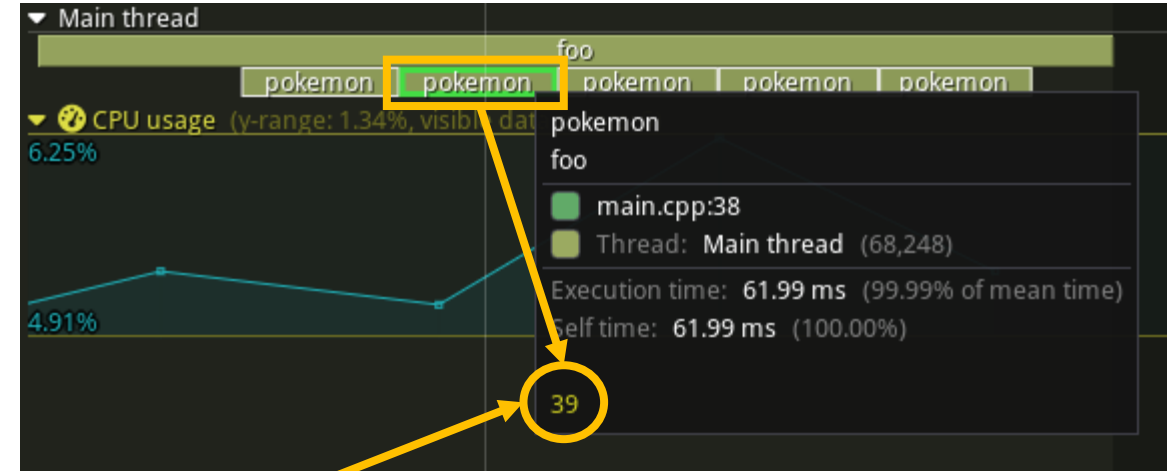
tracy::Color::



- **ZoneName**
 - **programmatically** change the zone name
 - can be **non-persistent string** (e.g., an `std::string` in the stack)
Avoid! (it will confuse people unfamiliar with your code)
 - new name **WON'T appear** in profiler statistics!
- **ZoneColor**
 - **programmatically** change the zone color

Tracy Zoning 201

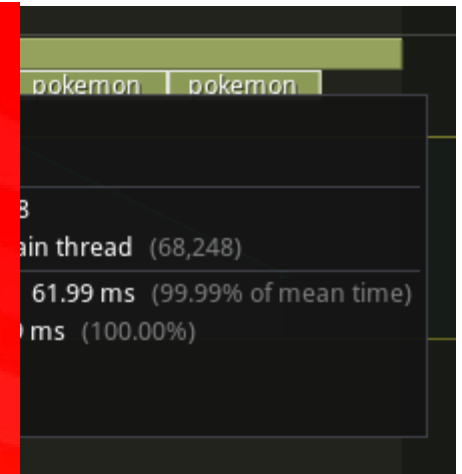
```
int foo()
{
    ZoneScoped;
    ...
    Pokedex pokedex;
    ...
    ZoneValue(pokedex.size());
    for (auto& pokemon : pokedex)
    {
        ZoneScopedN("pokemon");
        ZoneValue(pokemon.id);
        ...
    }
}
```



- **ZoneValue**
 - logs a value to the current zone scope
 - accepts ~~int64 or double~~ **uint64**

Tracy Zoning 201

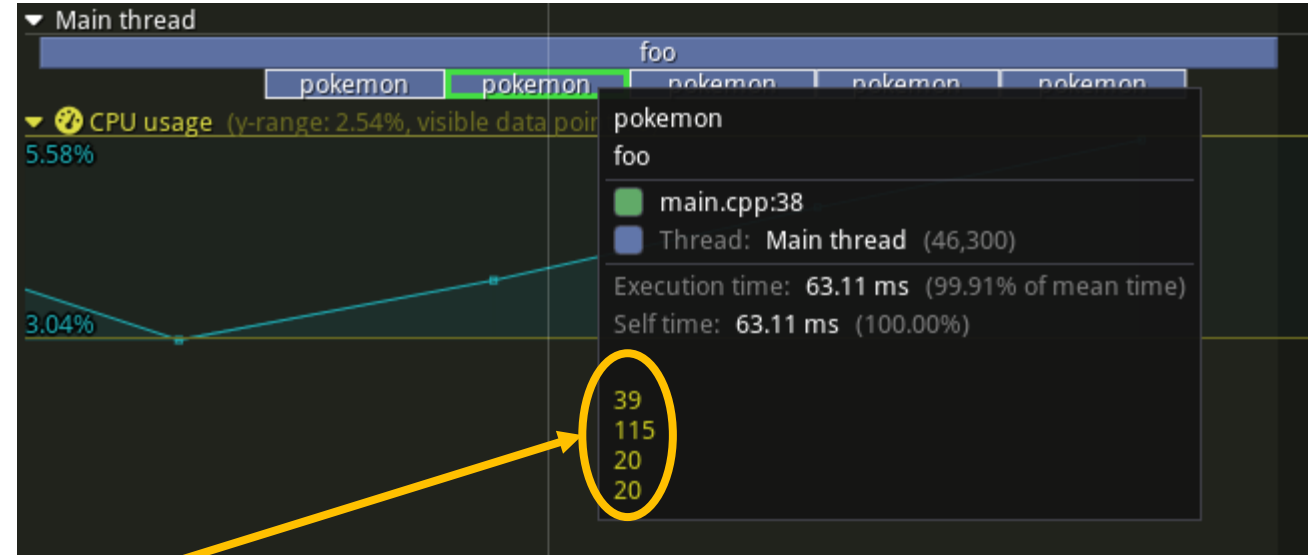
```
int foo()  
{  
    ZoneScoped;  
    ...  
    Pokedex pokedex;  
    ...  
    ZoneValue(pokedex  
for (auto& pokemo  
{  
    ZoneScopedN("  
    ZoneValue(pok  
    ...  
}  
}
```



ne scope

Tracy Zoning 201

```
int foo()  
{  
    ZoneScoped;  
    Pokedex pokedex;  
    for (auto& pokemon : pokedex)  
    {  
        ZoneScopedN("pokemon");  
        ZoneValue(pokemon.id);  
        ZoneValue(pokemon.hp);  
        ZoneValue(pokemon.def);  
        ZoneValue(pokemon.speed);  
        ...  
    }  
}
```



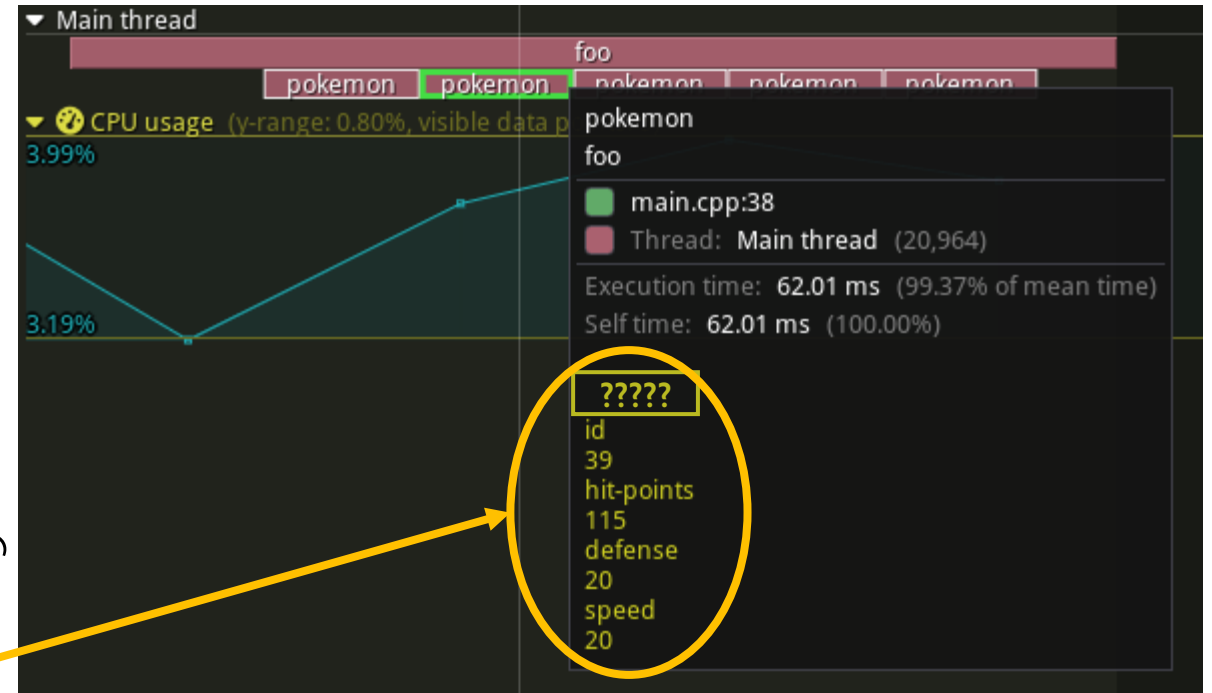
- **ZoneValue**
 - logs a value to the current zone scope
 - accepts ~~int64 or double~~ uint64
 - **multiple** values allowed!

Tracy Zoning 201

```
int foo()  
{
```

```
    ZoneScoped;  
    Pokedex pokedex;  
    for (auto& pokemon : pokedex)  
    {  
        ZoneScopedN("pokemon");  
        ...  
        ZoneText("hit-points", 10);  
        ZoneValue(pokemon.hp);  
        ZoneText("defense", 7);  
        ZoneValue(pokemon.def);  
        ...  
    }
```

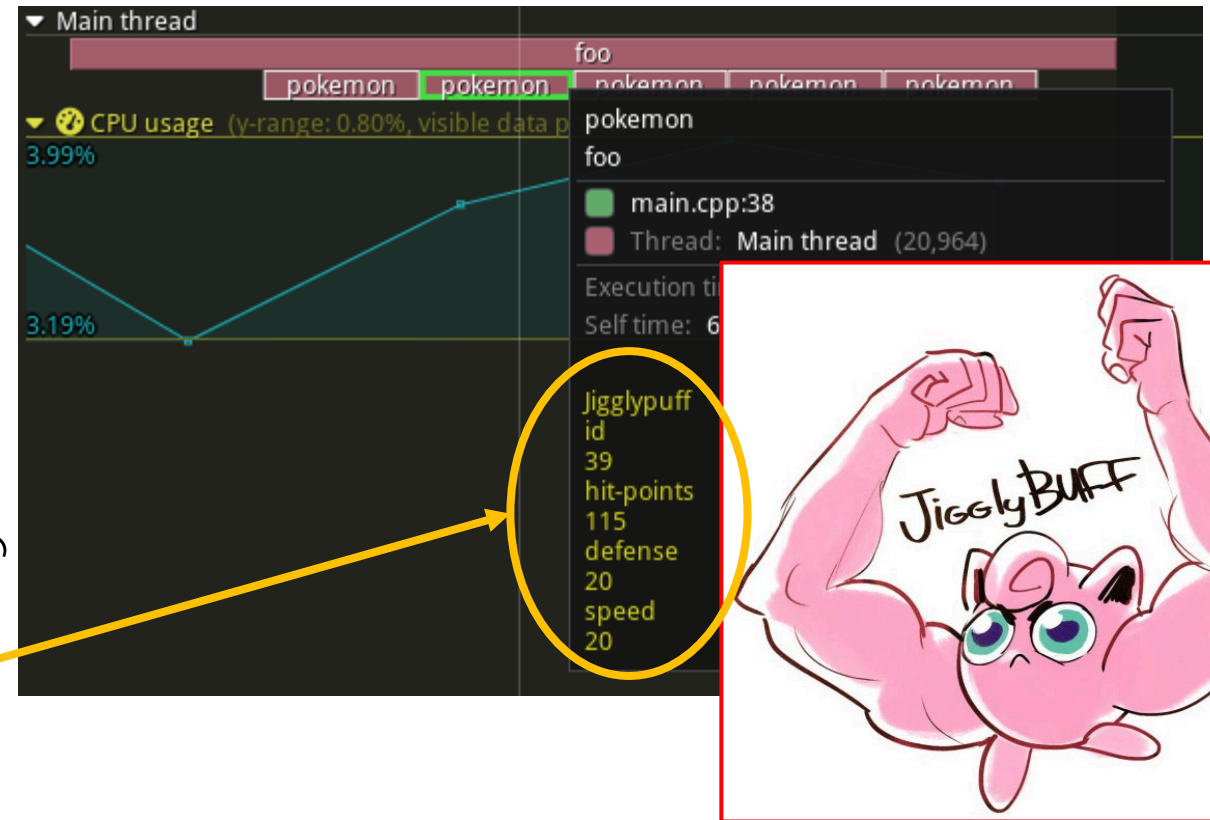
strlen("hit-points")



- **ZoneText**
 - logs a string to the current zone scope
 - non-persistent string allowed
 - multiple texts allowed!

Tracy Zoning 201

```
int foo()  
{  
    ZoneScoped;  
    Pokedex pokedex;  
    for (auto& pokemon : pokedex)  
    {  
        ZoneScopedN("pokemon");  
        ...  
        ZoneText("hit-points", 10);  
        ZoneValue(pokemon.hp);  
        ZoneText("defense", 7);  
        ZoneValue(pokemon.def);  
        ...  
    }  
}
```



Source: <https://www.reddit.com/r/pokemon/comments/3o5je7/jigglybuff/>

- **ZoneText**
 - **logs a string** to the current zone scope
 - **non-persistent string** allowed
 - **multiple** texts allowed!

Tracy Zoning 301

```
int foo()  
{  
    ZoneScoped;  
    ...  
    ZoneNamedN(initzone, "init", true);  
    ...  
    ZoneNamedN(setupzone, "setup", true);  
    ...  
    ZoneNamedN(simzone, "simulate", true);  
    ...  
    ZoneNamedN(logzone, "log", false);  
    ...  
    ZoneNamedN(cleanzone, "cleanup", true);  
    ...  
}
```



- **ZoneNamed[N|C]**
 - have **multiple zones** in the **same scope**
 - each "**zone identifier**" must be **unique** in that scope
 - creates a zone **stack**, **not a partition**
 - offers "**active**" argument to **toggle** (on/off)
a zone **rogrammatically**

Tracy Zoning 301

ZoneScoped[N | C | S]

ZoneColor(...)

ZoneValue(...)

ZoneText(...)

ZoneName(...)

ZoneIsActive()



ZoneNamed[N | C | S]

ZoneColorV(varname, ...)

ZoneValueV(varname, ...)

ZoneTextV(varname, ...)

ZoneNameV(varname, ...)

ZoneIsActiveV(varname)

Tracy Zoning 401

```
int foo()  
{  
    ZoneScoped;  
    Pokedex pokedex;  
    for (auto& pokemon : pokedex)  
    {  
        ZoneScopedN("pokemon");  
        ZoneName(pokemon.name, strlen(pokemon.name));  
        ...  
    }  
}
```

The screenshot shows the Tracy profiler interface. The 'Main thread' panel displays a timeline with zones for 'Pikachu', 'Jigglypuff', 'Meowth', 'Psyduck', and 'Onix'. The 'Statistics' panel shows a table of zones with columns for Name, L, Total time, Counts, and MTPC. The 'pokemon' zone is highlighted in the table.

Name	L	Total time	Counts	MTPC
pokemon	m	308.97 ms (36.58%)	5	61.79 ms
foo	m	109.04 ms (12.91%)	1	109.04 ms

Tracy Zoning 401

```
int foo()
{
    ZoneScoped;
    Pokedex pokedex;
    for (auto& pokemon : pokedex)
    {
        ZoneScopedN("pokemon");
        ZoneTransientN(pokezone, pokemon.name, true);
        ...
    }
}
```

Transient zones **may be necessary** when instrumenting a module that can be loaded **and unloaded** throughout the lifecycle of the host application (e.g., a plugin)

3.4.4 Transient zones

In order to prevent problems caused by unloadable code, described in section 3.1.1, transient zones copy the source location data to an on-heap buffer. This way, the requirement on the string literal data being accessible for the rest of the program lifetime is relaxed, at the cost of increased memory usage.

Main thread				
foo				
Pikachu Jigglypuff Meowth Psyduck Onix				
CPU usage				
Statistics				
Instrumentation Total zone count: 6 Visible zones: 6 Timing Self on				
Filter results Clear Limit range				
Name	L	Total time	Counts	MTPC
foo	rr	107.84 ms (12.64%)	1	107.84 ms
Psyduck	rr	61.17 ms (7.17%)	1	61.17 ms
Pikachu	rr	63.01 ms (7.39%)	1	63.01 ms
Onix	rr	63.82 ms (7.48%)	1	63.82 ms
Meowth	rr	62.99 ms (7.38%)	1	62.99 ms
Jigglypuff	rr	62.01 ms (7.27%)	1	62.01 ms

- ZoneTransientN

- name a zone with **non-persistent string**
- the **same zone** can have **multiple names**
- will **show up in statistics!**
- increased **runtime cost** to record zone

Tracy Zoning 501

```
g_main.cpp
game
421  /*
422  * Advances the world by 0.1 seconds
423  */
424  void
425  G_RunFrame(void)
426  {
427      ZoneScopedS(60);
428
429      int i;
430      edict_t *ent;
```

- **ZoneScopedS(depth)**
 - captures **callstack**
 - adds some **overhead**
 - useful for **code exploration**
 - **max** callstack depth: **~60**
 - (enables selective sampling profiling)
- **ZoneScoped[N|C|S]**

The screenshot shows the Tracy Zone info window for the function `G_RunFrame`. The `Call stack` button is highlighted. Below it, the Call stack window is open, showing the stack of frames. The `@ Frame location:` dropdown is set to `Source code`. The stack frames are listed in a table:

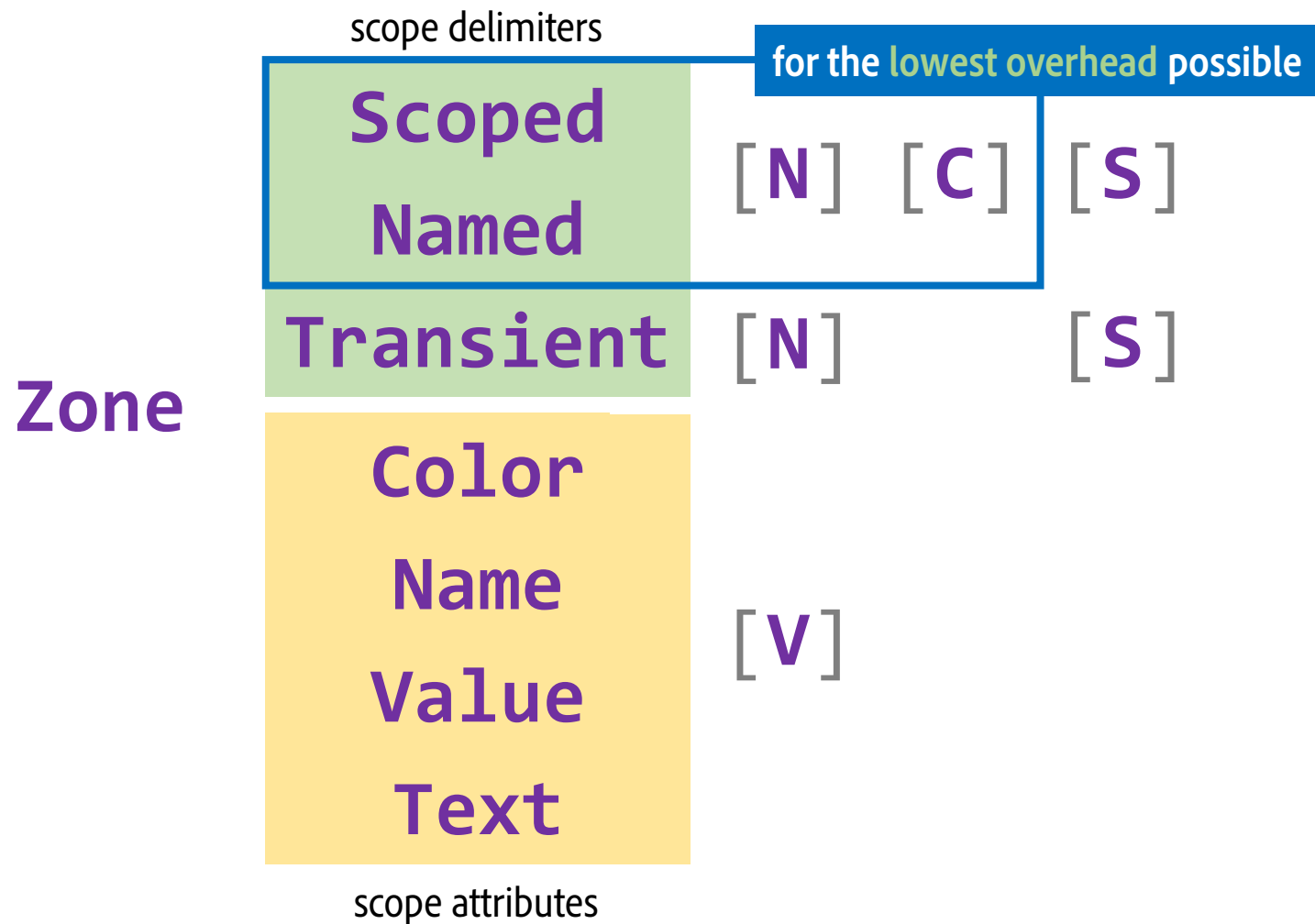
Frame	Function	Location	Image
inline	tracy::ScopedZone::ScopedZone	D:\ThirdParty\Tracy\public\client\TracyScoped.hpp:52	[game.dll]
0	G_RunFrame	D:\yquake2pp\src\game\g_main.cpp:427	[game.dll]
inline	SV_RunGameFrame	D:\yquake2pp\src\server\sv_main.cpp:359	[yquake2.exe]
1	SV_Frame	D:\yquake2pp\src\server\sv_main.cpp:429	[yquake2.exe]
2	Qcommon_Frame	D:\yquake2pp\src\common\frame.cpp:664	[yquake2.exe]
inline	Qcommon_Mainloop	D:\yquake2pp\src\common\frame.cpp:189	[yquake2.exe]
3	Qcommon_Init	D:\yquake2pp\src\common\frame.cpp:408	[yquake2.exe]
inline	CL_Quit_f	D:\yquake2pp\src\client\cl_main.cpp:316	[yquake2.exe]
inline	Qmain	D:\yquake2pp\src\backends\windows\main.cpp:125	[yquake2.exe]
4	main	D:\yquake2pp\src\backends\windows\main.cpp:139	[yquake2.exe]
5	_tmainCRTStartup	T:\Tools\zig\current\lib\libc\mingw\crt\crtexe.c:321	[yquake2.exe]
6	mainCRTStartup	T:\Tools\zig\current\lib\libc\mingw\crt\crtexe.c:204	[yquake2.exe]
7	BaseThreadInitThunk	[unknown]	[KERNEL32.DLL]
8	RtlUserThreadStart	[unknown]	[ntdll.dll]

Callstack capture cost

Depth	x86	x64	ARM	ARM64
1	34 ns	98 ns	6.62 µs	6.63 µs
2	35 ns	150 ns	8.08 µs	8.25 µs
3	36 ns	168 ns	9.75 µs	10 µs
4	39 ns	190 ns	10.92 µs	11.58 µs
5	42 ns	206 ns	12.5 µs	13.33 µs
10	52 ns	306 ns	19.62 µs	21.71 µs
15	63 ns	415 ns	26.83 µs	30.13 µs
20	77 ns	531 ns	34.25 µs	38.71 µs
25	89 ns	630 ns	41.17 µs	47.17 µs
30	109 ns	735 ns	48.33 µs	55.63 µs
35	123 ns	843 ns	55.87 µs	64.09 µs
40	142 ns	950 ns	63.12 µs	72.59 µs
45	154 ns	1.05 µs	70.54 µs	81 µs
50	167 ns	1.16 µs	78 µs	89.5 µs
55	179 ns	1.26 µs	85.04 µs	98 µs
60	193 ns	1.37 µs	92.75 µs	106.59 µs

Tracy Manual: Section 3.11 – Collecting Timestamps

Recap (cheat sheet)

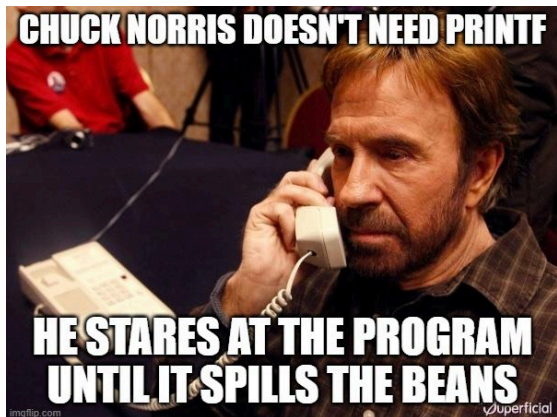


Suffixes:

- **N** : explicit name
- **C** : explicit color
- **S** : capture callstack

- **V** : explicit zone scope identifier
(when using **ZoneNamed**[N|C|S])
(or **ZoneTransient**[N|S])

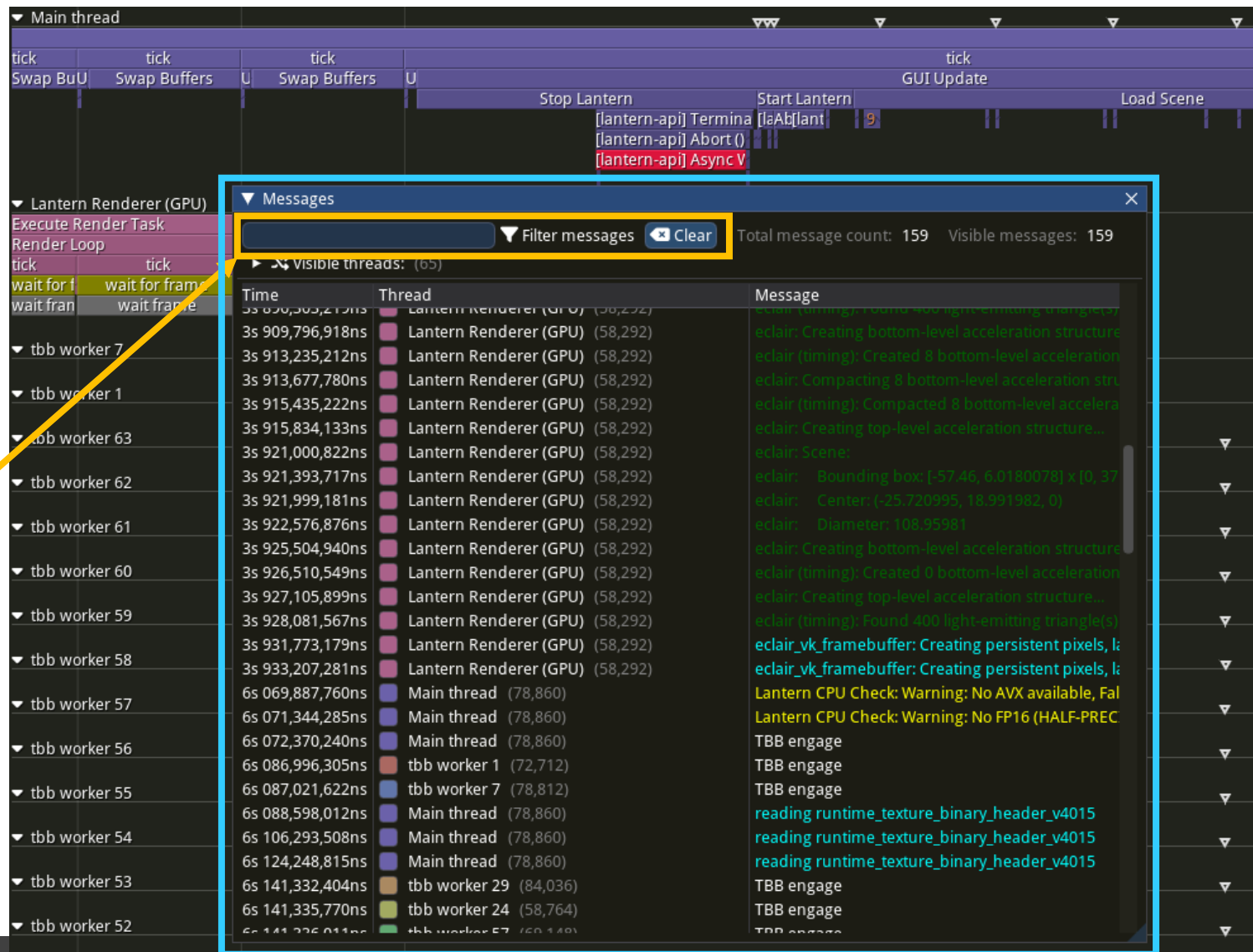
TracyMessage() is the new printf()



persistent string literal

TracyMessage[L | C | S]

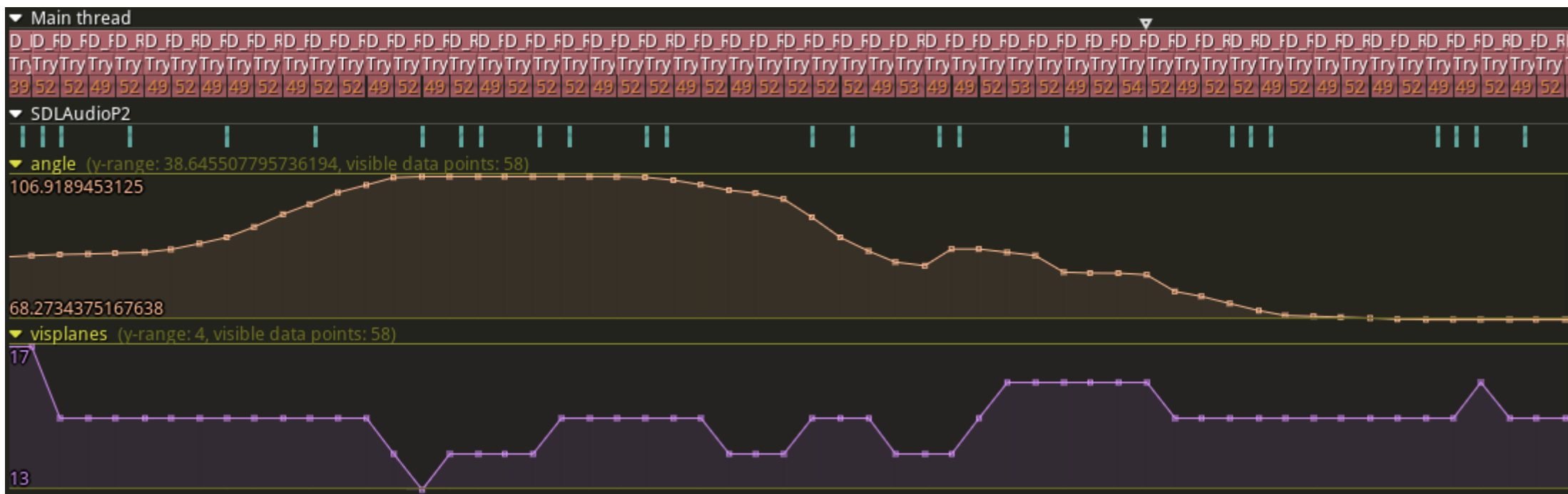
- console replacement!
- time-aware!
- thread-aware!
- message colors!
- browsable log!
- filterable log!
- no thread sync needed!



Plots

- `TracyPlot(name, value)`
persistent `int64, float`
string literal or `double`

plots are **shared** among threads
no per-thread plot segregation
but profiler **sorts on timestamp**



Plots

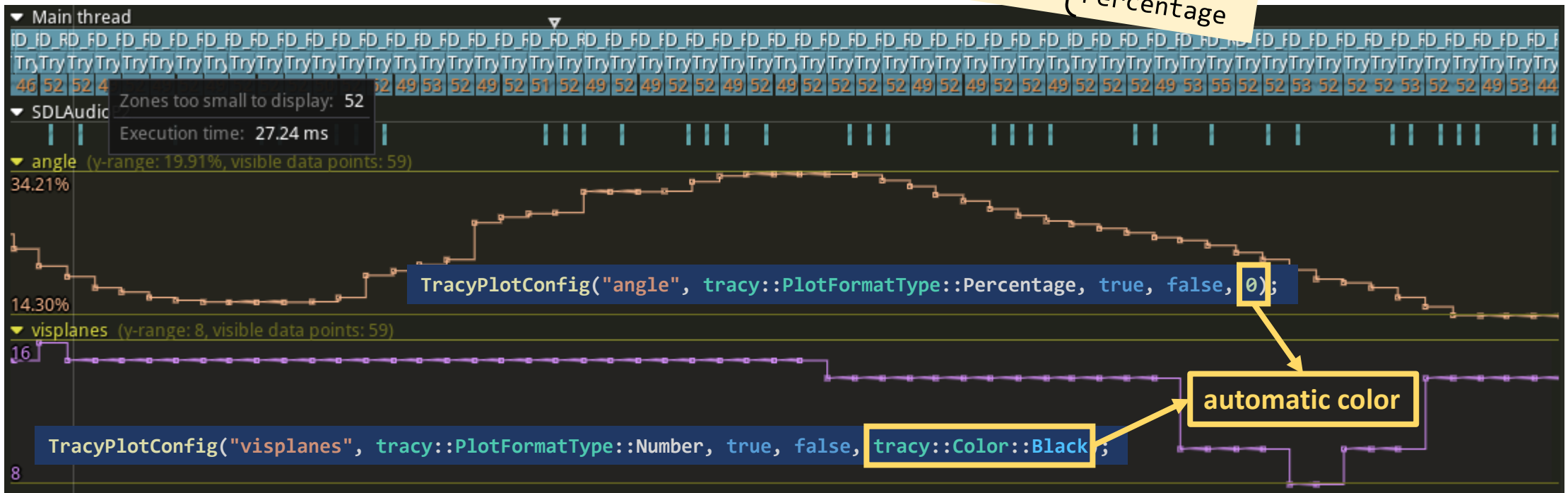
- `TracyPlotConfig(name, type, step, fill, color)`

`true`: stairstep
`false`: linear

`true`: shade area
`false`: outline only

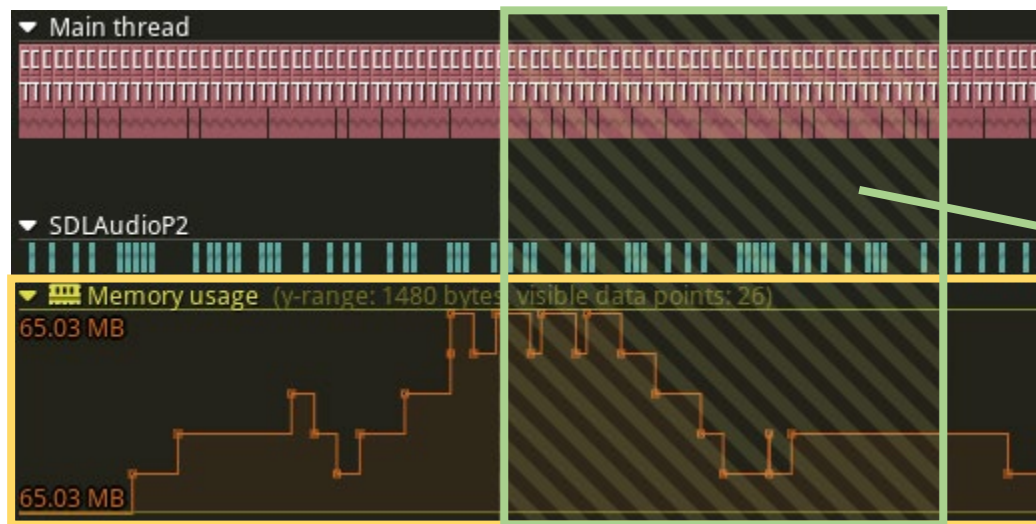
`tracy::Color::`

`tracy::PlotFormatType::` {
Number
Memory
Percentage



Tracking Memory Allocations

- `TracyAlloc[N|S](ptr, size, ...)`
- `TracyFree[N|S](ptr, ...)`



`TracyAlloc` and `TracyFree`
won't do memory **allocation**:
it's merely for **tracking**

The screenshot shows the Tracy memory tracking interface. At the top, there's a 'Memory' section with a dropdown menu for 'Memory pool' set to 'Default allocator'. Below it, there's a summary of memory usage: 'Total allocations: 18,472', 'Active allocations: 4,435', 'Memory usage: 65.03 MB', and 'Memory span: ...'. Below this, there's a table of 'Active allocations (4,435)' with columns: 'Address', 'Size', 'Appeared...', 'Duration', 'Thread', 'Zone alloc', and 'Zone free'. The table lists 15 active allocations, all from the 'Main thread' and in the 'doom_strdup' zone. A green box highlights the table, and a green arrow points from the 'Memory usage' graph in the left-hand screenshot to it.

Address	Size	Appeared...	Duration	Thread	Zone alloc	Zone free
0x2657fa3d5f	6 bytes	275,887,006ns	22.66 s	Main thread	doom_strdup	active
0x2657fa56af	24 bytes	275,918,470ns	22.66 s	Main thread	doom_malloc	active
0x2650c0f504	32 MB	3s 756,820,177	19.18 s	Main thread	doom_malloc	active
0x2650b9a94a	0 bytes	3s 756,900,165	19.18 s	Main thread	doom_strdup	active
0x2650bf681f	11 bytes	3s 757,173,674	19.18 s	Main thread	doom_strdup	active
0x2650bf682f	15 bytes	3s 757,223,345	19.18 s	Main thread	doom_strdup	active
0x2650b9a957	2 bytes	3s 757,446,604	19.18 s	Main thread	doom_strdup	active
0x2650bf6855	23 bytes	3s 757,454,096	19.18 s	Main thread	doom_strdup	active
0x2650b9a925	7 bytes	3s 757,455,801	19.18 s	Main thread	doom_strdup	active
0x2650be6de2	25 bytes	3s 757,458,045	19.18 s	Main thread	doom_strdup	active
0x2650b9a926	5 bytes	3s 757,459,964	19.18 s	Main thread	doom_strdup	active
0x2650bf6807	9 bytes	3s 757,461,717	19.18 s	Main thread	doom_strdup	active
0x2650bf6831	21 bytes	3s 757,463,483	19.18 s	Main thread	doom_strdup	active
0x2650bf6833	10 bytes	3s 757,465,088	19.18 s	Main thread	doom_strdup	active

Lock instrumentation

```
std::mutex my_mutex;
```



```
TracyLockable(std::mutex, my_mutex);
```

and then

```
std::lock_guard<LockableBase(std::mutex)> lock (my_mutex);
```



There's also **TracySharedLockable[N]** (and **SharedLockableBase**)
for readers-writer mutex (`std::shared_mutex`)

What's Next?

Tips & Tricks
(& Gotchas)

Study Cases
from Adobe

Explore
Together

&

Time for
Questions

GPU
instrumentation
(very briefly)

A Look at Tracy's
Sampling Mode

Tips & Tricks

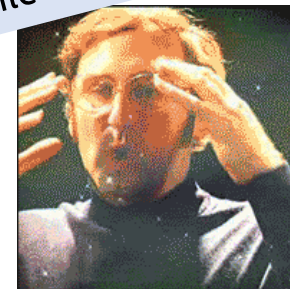


Apple users...

...get a **real mouse***!



holding right button
while moving pointer!



*or help fix Apple trackpad gestures in ImGui

Tips & Tricks

Instrument the “usual suspects”

- Idling usually a good thing!
- Waits implicit ones too: e.g. `future.get()`
- Sleeps please, just don't...
- Lock acquisition (contention) ~~Tracy~~`Lockable`
- Memory operations `memcpy()`, `malloc()` et al.
- Explicit system calls (or heavy API calls) shader compilation
- Big-O activity (high-level) sorting, searching, container manipulations, ...
- I/O activity file, network, logging, ...
- Unpredictable events interrupts, user input activity, spurious wake-ups, ...

Tips & Tricks

Color zones appropriately

- | | |
|--|--|
| • Idling | usually a good thing! |
| • Waits | <i>implicit</i> ones too: e.g. <code>future.get()</code> |
| • Sleeps | better yet, avoid <code>sleep()</code> in production code! |
| • Lock acquisition (contention) | TracyLockable |
| • Memory operations | <code>memcpy()</code> , <code>malloc()</code> et al. |
| • Explicit system calls (or heavy API calls) | shader compilation |
| • Big-O activity (high-level) | sorting, searching, container manipulations, ... |
| • I/O activity | file, network, logging, ... |
| • Unpredictable events | interrupts, user input activity, spurious wake-ups, ... |

Tips & Tricks

Color zones appropriately

- Idling

- Waits

- Sleep

- Lock

- Mem

- Explicit

- Big-O

- I/O act

- Unpredictable events

good thing!

figure.get()

ion code!

lockable

() et al.

compilation

g, searching, container manipulations, ...

file, network, logging, ...

interrupts, user input activity, spurious wake-ups, ...

Just to be clear...

Rely on Tracy's **AUTOMATIC** coloring **EVERYWHERE**
EXCEPT for these **KNOWN-TO-BE-PROBLEMATIC** cases
(they **usually** appear at the **bottom** of the **zone stack**)
pick **vibrant colors**: you want them to **pop up** in the profiler!
(Tracy's automatic coloring never chooses such saturated colors)

Tips & Tricks

Standardize the colors (don't think about them!)

Semantic Palette

```
enum Tier
{
    Idle    = tracy::Color::DimGray,
    Wait    = tracy::Color::Crimson,
    Sleep   = tracy::Color::Tomato,
    Lock    = tracy::Color::Coral,
    Memory  = tracy::Color::Goldenrod,
    System  = tracy::Color::Gold,
    BigO    = tracy::Color::LimeGreen,
    IO      = tracy::Color::Blue,
    Fickle  = tracy::Color::Magenta,
    ...
};
```

X11 color names

(consider color blindness!)



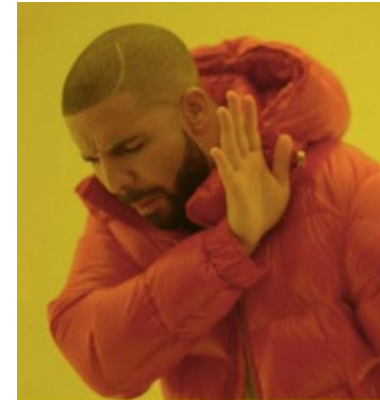
Tips & Tricks

Name your threads!

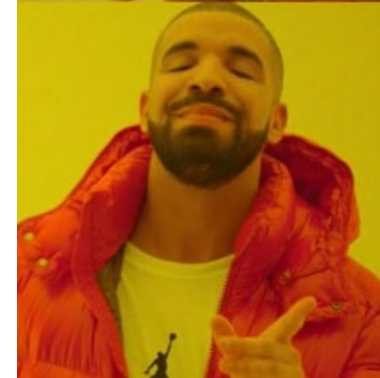
```
tracy::SetThreadName("thread-name")
```

TBB workers : use task_scheduler_observer

```
class tracy_tbb_observer : public oneapi::tbb::task_scheduler_observer {
public:
    tracy_tbb_observer(tbb::task_arena& arena) : oneapi::tbb::task_scheduler_observer(arena) {
        observe(true); // activate the observer
    }
    void on_scheduler_entry(bool is_worker) override {
        if (is_worker) {
            int tid = tbb::this_task_arena::current_thread_index();
            tracy::SetThreadName(std::string("tbb worker #").append(std::to_string(tid)).c_str());
        }
    }
    void on_scheduler_exit(bool is_worker) override { }
};
```



▼ 8048781
process_task
▼ 8048778
process_task
▼ 8048779
process_task
▼ 8048777
process_task
▼ Main thread
main



▼ tbb worker3
process_task
▼ tbb worker2
process_task
▼ tbb worker1
process_task
▼ tbb worker0
process_task
▼ Main thread
main

Tips & Tricks

Consider “**siloeing**” the **usual suspects**

example: don't call **memcpy()** directly... call some **my_memcpy()** which then calls **memcpy()**

```
void* my_memcpy(void* dst, const void* src, size_t size)
{
    ZoneScopedC(tracy::Color::Blue);
    return ::memcpy(dst, src, size);
}
```

unified point for instrumenting the usual suspects!

whole-program **symbol hijacking**

(e.g., to replace malloc without siloeing across all process modules)

[LD PRELOAD](#)

[DYLD INSERT LIBRARIES](#)

[Microsoft Detours](#)

[weak symbol linkage interposition](#)

[MSVC linker /alternatename](#)



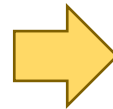
Tips & Tricks

Instrumenting **dynamic dispatch** (e.g., virtual calls)

Need to **instrument all overrides...**

Consider using **static delegate** interface to call the **virtual** method
(if only we could manipulate/alter vtable entries... C-style (or COM-style) dispatch tables?)

```
class Shape
{
    public:
        virtual float area();
};
```



```
class Shape
{
    public:
        float area() {
            ZoneScoped;
            return calculate_area();
        }
    protected:
        virtual float calculate_area();
};
```

Tips & Tricks

Capture **callstack** on **"terminal"** zones for **"code reconnaissance"**

```

648 //
649 int
650 FS_LoadFile(char *path, void **buffer)
651 {
652     ZoneScopedCS(tracy::Color::Blue, 64);
653     ZoneText(path, strlen(path));
654
655     byte *buf; /* Buffer. */
656     int size; /* File size. */
657     fileHandle_t f; /* File handle. */
658

```

▼ Main thread

- Qcommon_Frame
- CL_Frame
- SCR_UpdateScreen
- FS_LoadFile
- Q_fq_frQ_foQ_foQ_fopQ_fop
- Q_fopen
- QFS_

▼ 🧠 CPU usage (y-range: 1.79%, visible data points: 2)

8.04%

- the “usual suspects” are usually leaf/terminal calls
- they appear at the bottom of the zone stack
- ideal candidates for code recon via callstack inspection



Tips & Tricks

Capture **callstack** on **allocation** to hunt down **leaks** : **TracyAlloc[N]S()**

The screenshot shows the Tracy Memory tool interface. At the top, it displays summary statistics: Total allocations: 4,218, Active allocations: 1,058, Memory usage: 25.37 MB, and Memory span: 420.34 MB. Below this, the '@ Allocations' section has a search bar and 'Find'/'Clear' buttons. The 'Active allocations (6)' section shows a table of memory usage with 5070 bytes. The table columns are Address, Size, Appeared at, Duration, Thread, Zone alloc, Zone free, and Call stack. The 'Call stack' column for the first allocation is highlighted, showing a list of functions: 0. Qmalloc [ref_soft.dll], 1. R_LoadPic [ref_soft.dll], 2. R_LoadImage [ref_soft.dll], 3. RE_Draw_FindPic [ref_soft.dll], 4. M_DrawCursor [yquake2.exe], 5. M_Main_Draw [yquake2.exe], 6. M_Draw [yquake2.exe], 7. SCR_UpdateScreen [yquake2.exe], 8. CL_Frame [yquake2.exe], 9. Qcommon_Frame [yquake2.exe], 10. Qcommon_Mainloop [yquake2.exe], 11. Qcommon_Init [yquake2.exe], 12. CL_Quit_f [yquake2.exe], 13. Qmain [yquake2.exe], 14. main [yquake2.exe], 15. __tmainCRTStartup [yquake2.exe], 16. mainCRTStartup [yquake2.exe], 17. BaseThreadInitThunk [KERN], 18. RtlUserThreadStart [ntdll.dll].

Address	Size	Appeared at	Duration	Thread	Zone alloc	Zone free	Call stack
0x15d1d16ebc	845 bytes	4s 200,359,212	1.02 s	Main thread	Qmalloc	Qfree	alloc [free]
0x15d1d6061f	845 bytes	4s 201,298,125	1.02 s	Main thread	Qmalloc	Qfree	alloc [free]
0x15d1d60655	845 bytes	4s 202,266,676	1.02 s	Main thread	Qmalloc	Qfree	alloc [free]
0x15d1d6068b	845 bytes	4s 203,251,339	1.02 s	Main thread	Qmalloc	Qfree	alloc [free]
0x15d1d57c50	845 bytes	4s 204,230,647	1.02 s	Main thread	Qmalloc	Qfree	alloc [free]
0x15d1d57c86	845 bytes	4s 205,185,405	1.02 s	Main thread	Qmalloc	Qfree	alloc [free]

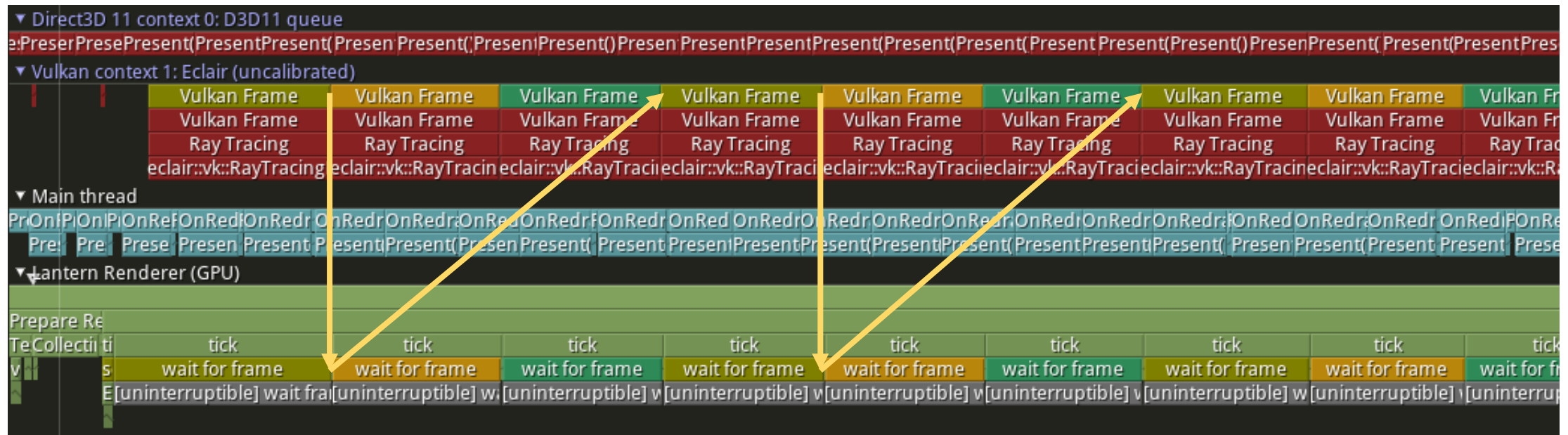
Below the table, there are options for 'Memory map', 'Bottom-up call stack tree', and 'Top-down call stack tree'.

For **known object leaks**, consider instrumenting **constructors** and **destructors** with **TracyAlloc/Free**



Tips & Tricks

Use color cycling to visually correlate zones



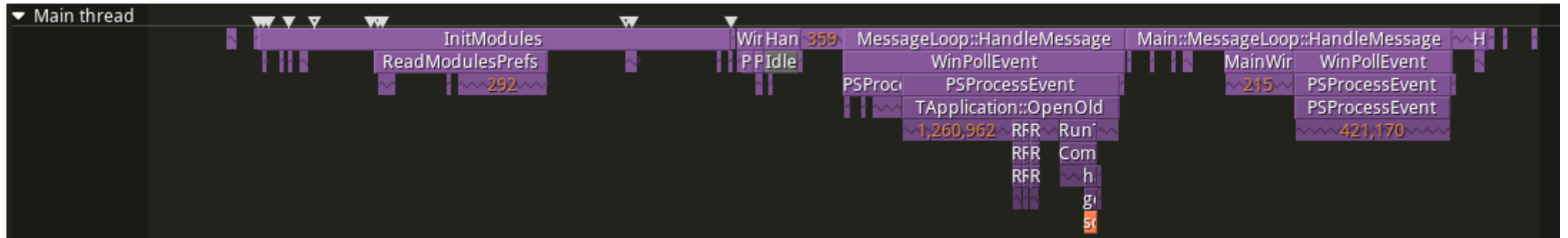
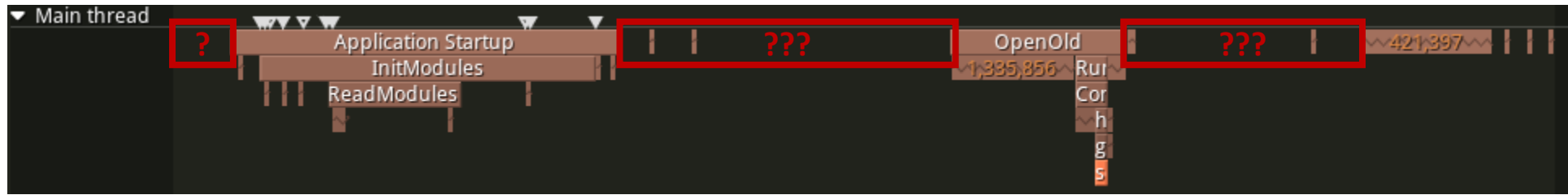
Tips & Tricks

Use `TracyIsConnected()` to **branch** on “**expensive**” tracing stuff

```
if (TracyIsConnected()) {  
    // expensive stuff I want to see in the profiler  
    // (now that I know the profiler is connected to the host app)  
    // ...  
    ZoneScopedS(60);    // let's capture the callstack  
    // ...  
    ZoneText(...);  
    // ...  
    TracyMessage(...);  
    // ...  
}
```

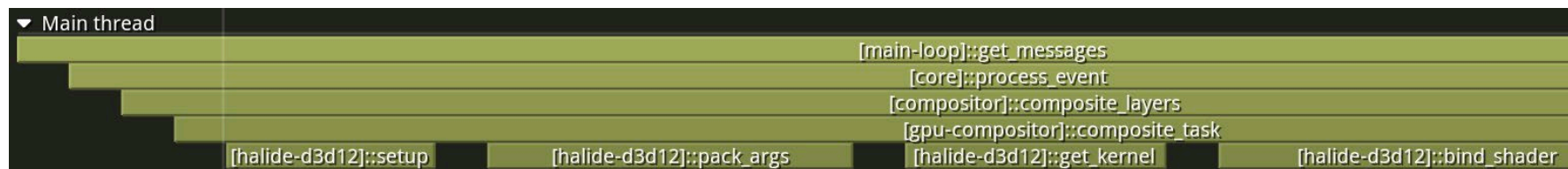
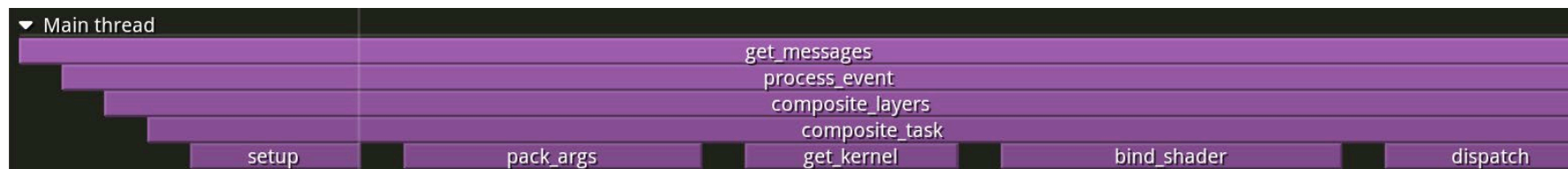

Tips & Tricks

Avoid “**empty gaps**” between zones (at least in the **top-most** level)



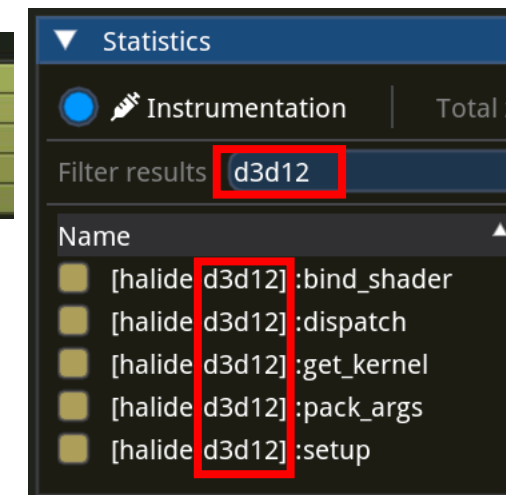
Tips & Tricks

Add **API prefix** to zone name when no such prefix/namespace exists



useful for **filtering** in statistics window

Use the **:: separator** !
Tracy Profiler knows about it and
will **"split"** on it for visualization



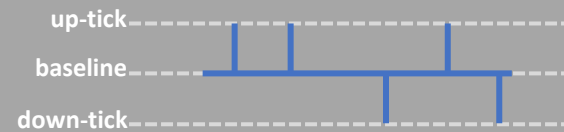
Tips & Tricks

Use **tick signals** to visualize **critical activity**

- user input
- critical callback
- interrupt signals
- IPC signals
- completion signals
- wake-up signals

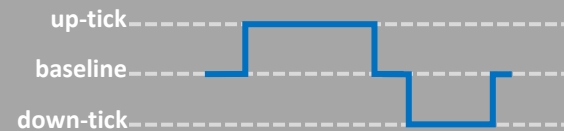
`TracyPlotConfig(name, type, stairstep, true, fill, color)`

pulse (heartbeat) plots

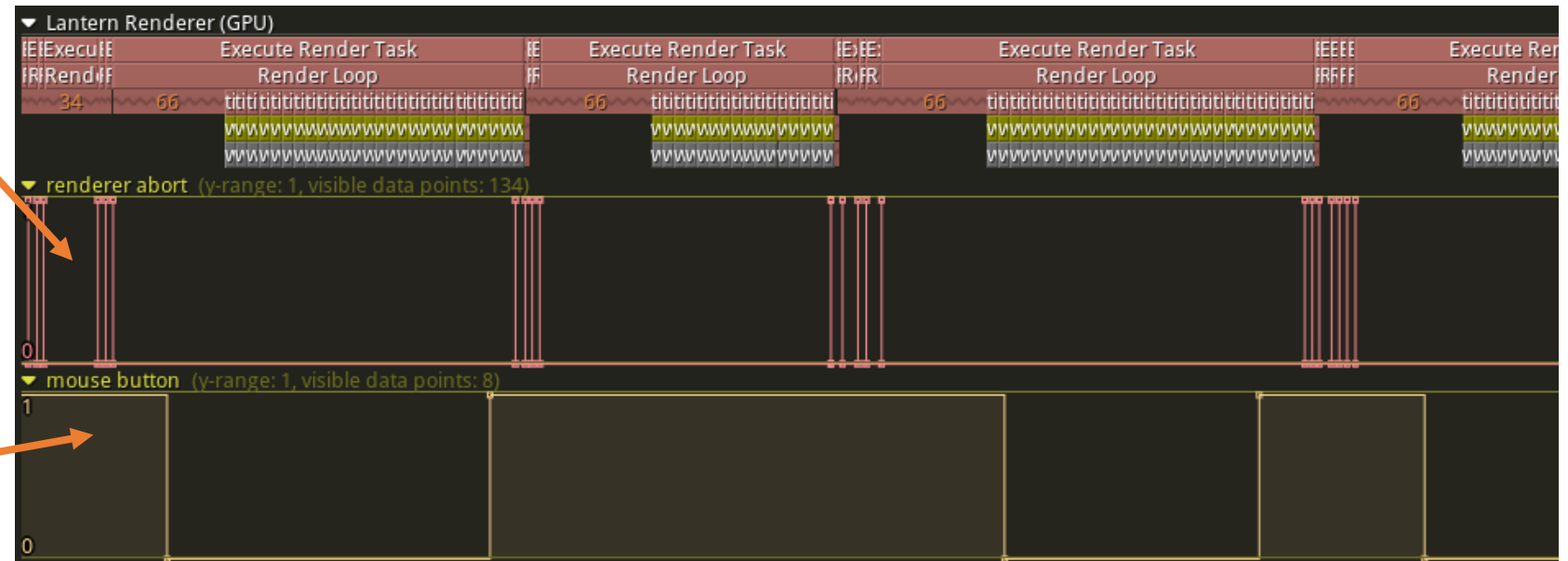


```
TracyPlot(name, tick);  
TracyPlot(name, baseline);
```

clock (sustain) plots



```
TracyPlot(name, tick);  
// ... activity ...  
TracyPlot(name, baseline);
```

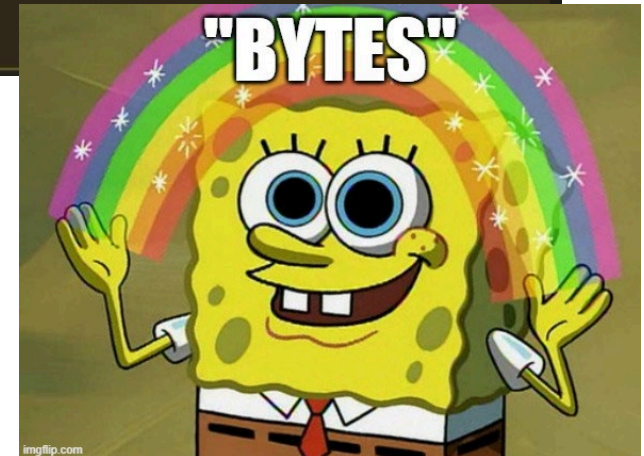
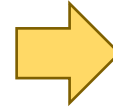


Tips & Tricks

Use `TracyAlloc()` and `TracyFree()` to *track anything* (objects, resources, handles, ...)

```
TracyAllocN(&frodo, 1, "party members");
TracyAllocN(&gandalf, 1, "party members");
TracyAllocN(&aragorn, 1, "party members");
// ...
TracyMessageL("[Gandalf] You shall not pass!");
TracyFreeN(&gandalf, "party members");
```

bytes...



Tips & Tricks

Instrumenting lock contention

too intrusive...

```
std::mutex my_mutex;
```



```
TracyLockable(std::mutex, my_mutex);
```

and then

```
std::lock_guard<LockableBase(std::mutex)> lock (my_mutex);
```

do this **siloining** instead!

```
[[nodiscard]]  
auto scoped_mutex_lock(std::mutex& m) {  
    ZoneScopedC(tracy::Color::Coral);  
    return std::unique_lock(m);  
}
```

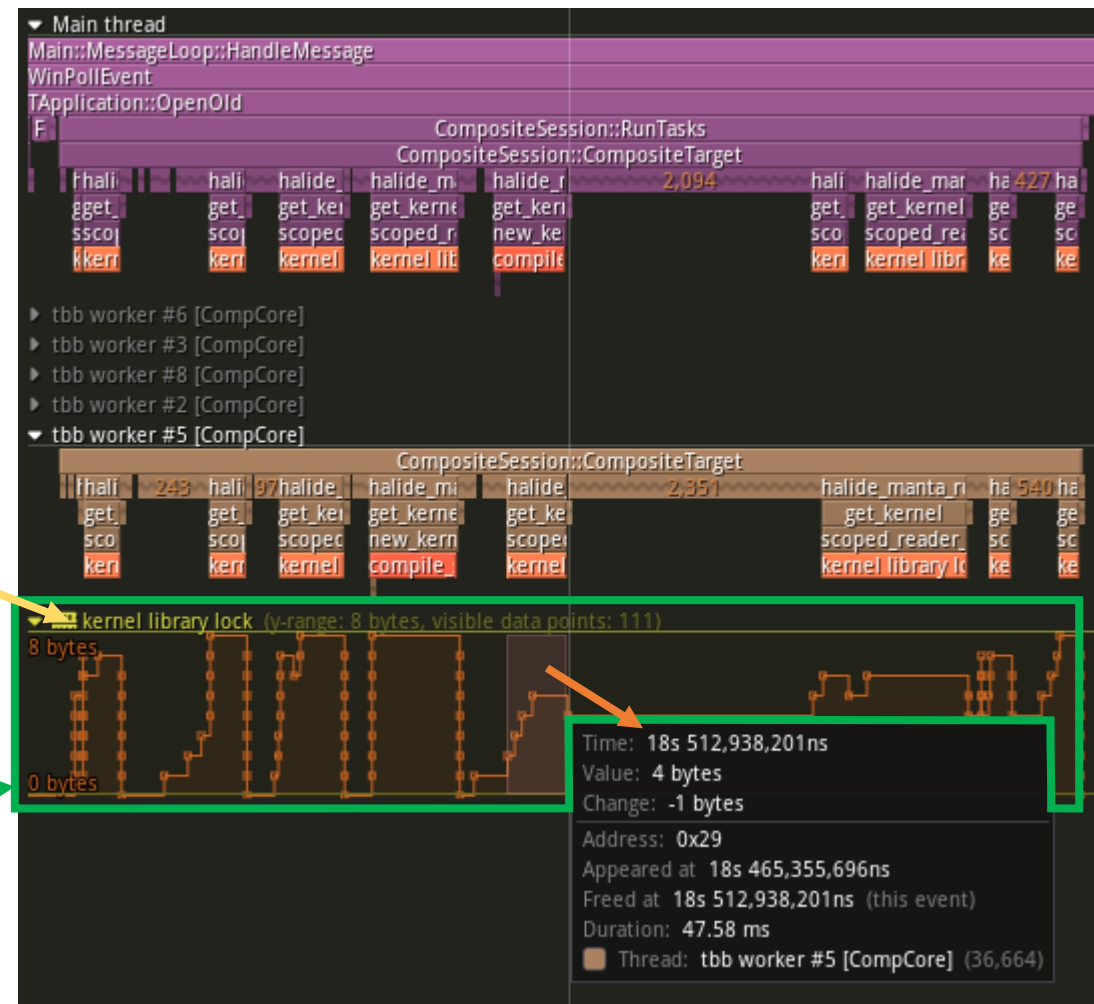
easily adaptable to reader-writer mutex as well
(std::shared_mutex and std::shared_lock)

Tips & Tricks

Measuring lock contention

```
static std::atomic<int> lock_id_generator = 0;

[[nodiscard]]
auto scoped_mutex_lock(std::mutex& m,
                      const char* name = "scoped_mutex_lock")
{
    if (m.try_lock())
    {
        // lucky! no contention!
        std::unique_lock lock (m, std::adopt_lock);
        return lock;
    }
    // uh-oh... possible contention...
    ZoneTransientN(mutex_lock_scope, name, true);
    ZoneColorV(mutex_lock_scope, tracy::Color::Coral);
    auto token = reinterpret_cast<void*>(&lock_id_generator);
    TracyAllocN(token, 1, name);
    std::unique_lock lock (m);
    TracyFreeN(token, name);
    return lock;
}
```



Tips & Tricks

Begin/End (and track) **zone** on **different scopes** or **across threads**

e.g.: task starts in main thread, moves between worker threads, and gets retired on a reclaimer thread

- A few **palliative** workarounds:
 - Use **FrameMarkStart** and **FrameMarkEnd**
 - Use “matching” **TracyMessage**’s
 - Use **stairstep TracyPlot** (uptick/downtick)
 - Assign unique **ZoneColor** to each task
- My **recommendation**: use **TracyAlloc** and **TracyFree**
(it’s basically a **stackable stairstep plot**!)

Tips & Tricks

Have an in-app **GUI** element to **launch & connect** the Tracy profiler

Team must agree on a **common location** for the Tracy profiler executable

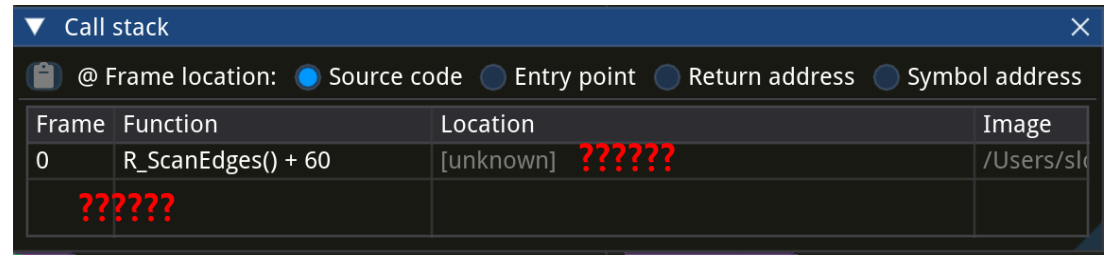
Md
David Farrell's
pro-tip!



Gotchas

Where are my Missing callstack symbols

- Compile with debug symbols
 - [gcc/clang] -g
 - [MSVC] /DEBUG
- DON'T omit frame pointers
 - [gcc/clang] -fno-omit-frame-pointer
 - [MSVC] not applicable for 64bit toolchain (/Oy- for 32bit toolchain)
- Make host executable symbols available to the dynamic linker
 - [MacOS] -export_dynamic
 - [Linux] -rdynamic or --export-dynamic (or -export_dynamic)
 - [Windows] not applicable (--export-dynamic is available in gcc/clang for WSL and MinGW, but no effect on PE executables, afaik)
- [MacOS] Make sure to call dsymutil on the executable *and* any dylib linked to it



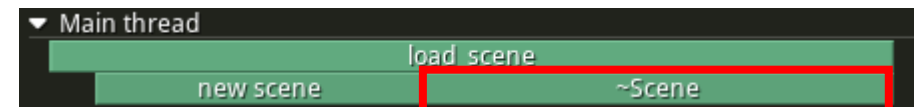
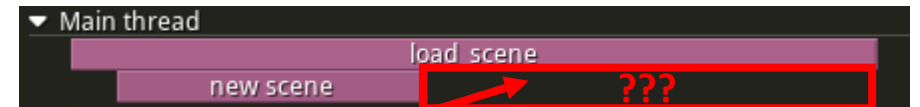
Refer to Tracy Manual: Section 3.11.1 – Debugging Symbols

Gotchas

Unexplainable “tail” gaps:

- Implicit **deferred execution** (e.g. smart-pointer deleting an object)
- Instrument expensive **destructors**
- or **explicitly release** smart objects at the end of a scope

```
bool Viewport::load_scene(const char* filename)
{
    ZoneScoped;
    // ...
    std::shared_ptr<Scene> new_scene;
    {
        ZoneScopedN("new scene");
        new_scene = std::make_shared<Scene>(filename);
        if (!new_scene) return false;
    }
    m_scene = new_scene;
    return true;
}
```

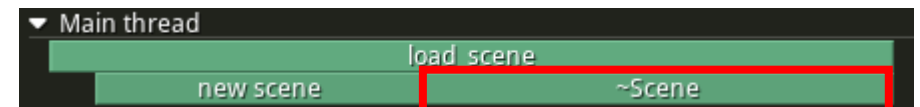
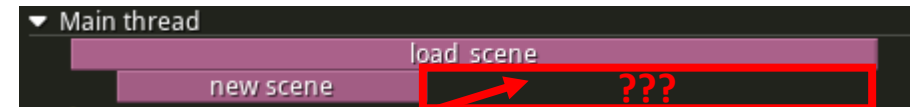


Gotchas

Unexplainable “tail” gaps:

- Implicit **deferred execution** (e.g. smart-pointer deleting an object)
- Instrument expensive **destructors**
- or **explicitly release** smart objects at the end of a scope

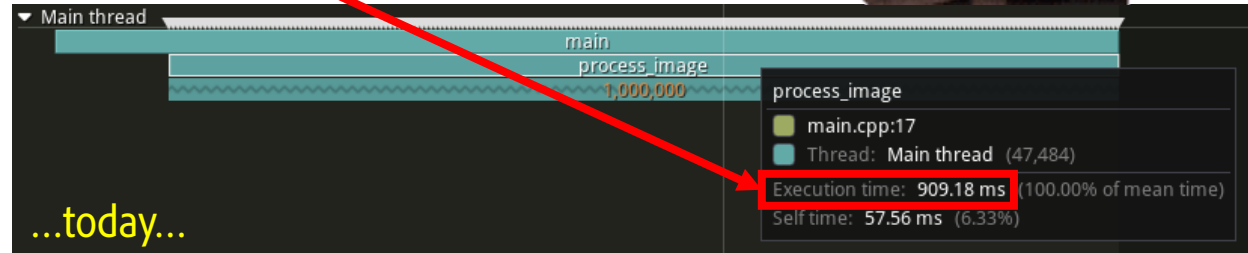
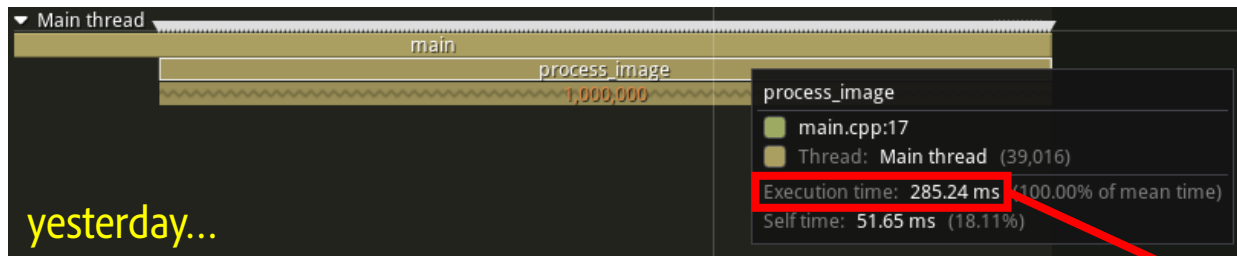
```
bool Viewport::load_scene(const char* filename)
{
    ZoneScoped;
    // ...
    std::shared_ptr<Scene> new_scene;
    {
        ZoneScopedN("new scene");
        new_scene = std::make_shared<Scene>(filename);
        if (!new_scene) return false;
    }
    m_scene = new_scene;
    return true;
}
```



Gotchas

Zones are **suddenly** taking a lot **more time**

- Have you recently switched to a **Debug build**?
- Only profile **Release builds** (preferably **detached from debugger**)
- Run profiler tool on a **different machine**
- Have you since started **capturing callstacks** (too frequently)?



Gotchas

Beware **excessive** instrumentation!

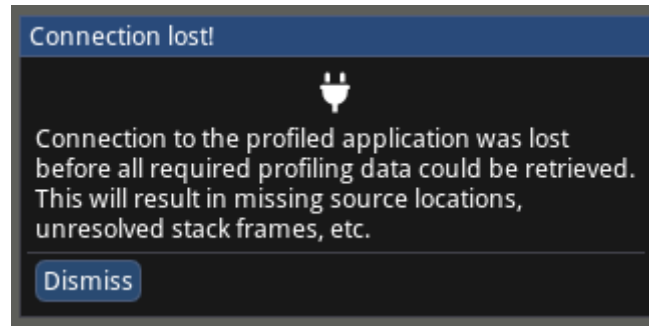
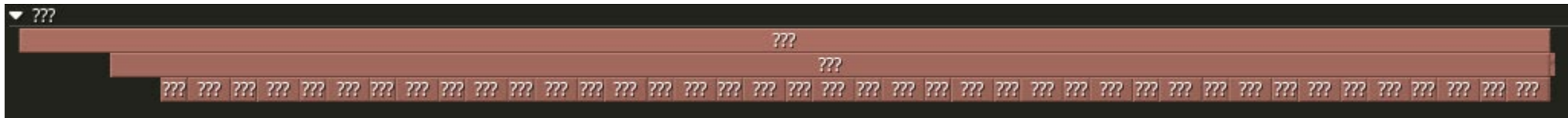
- hot **inner-loops**, small/inlined routines called frequently, ...
- **Profiling skew**: when profiling itself starts affecting performance
- May also affect **compiler optimizations**!

skewed profiling is still **useful**,
but requires a **different mindset**

Gotchas

Profiler **not showing the name** of some zones: getting **???** instead

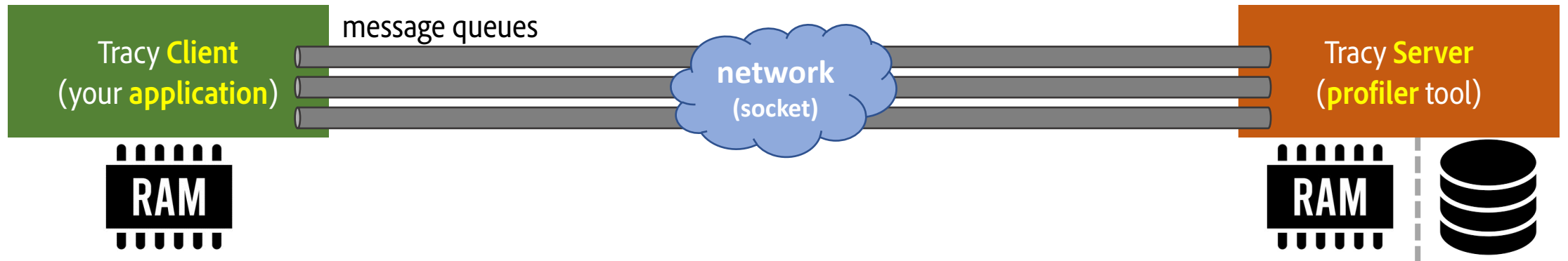
- just **wait** a little bit more (host application may not have sent this data yet)
- the host application may have **crashed** or terminated abruptly
- ensure zone name is a **persistent string**, or use **ZoneTransient()**
- if zone belongs to **plugin code**, make sure the plugin has **not been unloaded**



Gotchas

High RAM Usage

- Tracy **never drops** events! (there's no circular/LRU event queue)
- If profiler is **not connected**, application **memory keeps growing!**
- If profiler is running on the **same machine**, it is **competing for RAM**

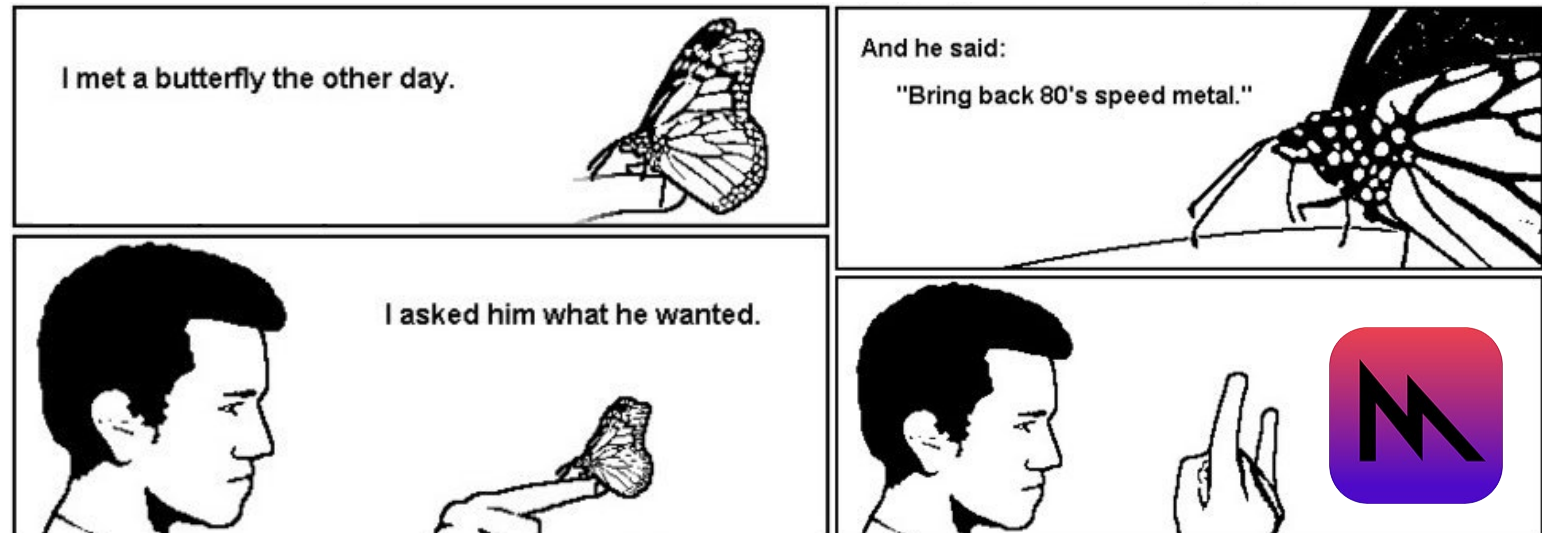


- Tip: #define **TRACY_ON_DEMAND** to delay producer until profiler gets connected
- Tip: use **capture** utility to “**sink**” Tracy events to **disk**
- Tip: consider running the profiler GUI on a **different machine**

Known Issues

macOS/iOS Limitations

- Profiler may get **interrupted** when **application finishes**
 - It's Apple, not Tracy! Ongoing workaround discussion: <https://github.com/wolfpld/tracy/issues/8>
- Vertical scrollbar missing on main profiler window
 - It's ImGui, not Tracy! <https://github.com/wolfpld/tracy/issues/631>
- No GPU **Metal API instrumentation** yet
 - I'm working on it!



Known Issues

Program **hangs** when calling **exit()**

- Try **TRACY_MANUAL_LIFETIME**
- or just **don't call exit()** or similar :-)



Known Issues

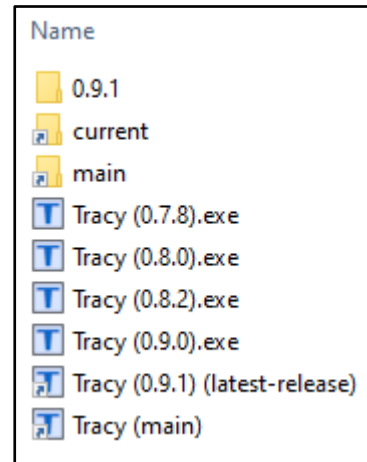
Tracy **capture files** are **not backwards compatible**

- Try using the **update** utility to convert trace files

download pre-built binary (Windows-only):
<https://github.com/wolfpld/tracy/releases>

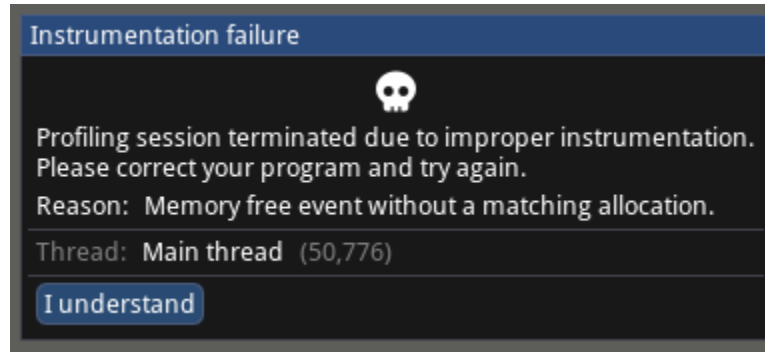
or build it yourself:
<tracy-repo>/update/build

- or **keep older versions** of Tracy around



Known Issues

`TracyFree[N|S]()` may **disconnect** profiler if **no matching allocation** exists

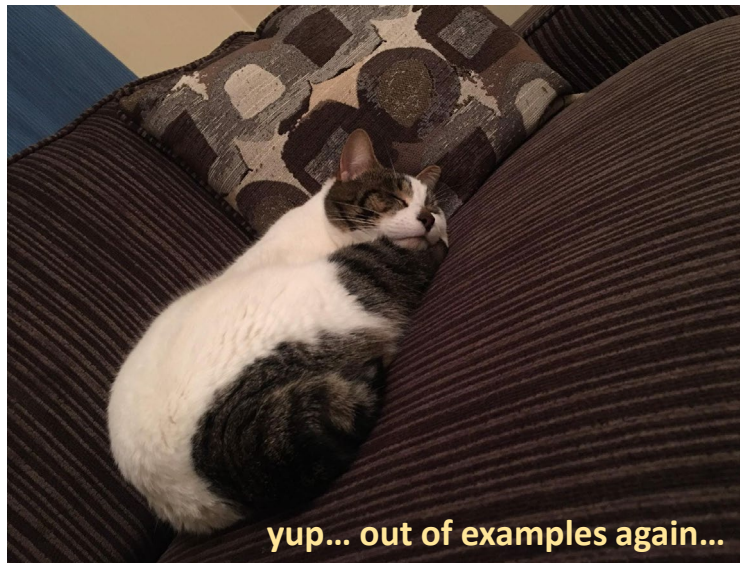


- Workaround:
 - force `m_ignoreMemFreeFaults` to `true` in `TracyWorker.cpp`
 - then make sure to **rebuild** the profiler tool

Known Issues

My program **won't connect** to the profiler

- may be the case for **short-lived** programs
- ... that said, you may `#define TRACY_NO_EXIT` in the host program (client) to **prevent** it from exiting until a **connection** to the profiler is established



yup... out of examples again...

More Limitations

2.1.8 Limitations

When using Tracy Profiler, keep in mind the following requirements:

- The application may use each lock in no more than 64 unique threads.
- There can be no more than 65534 unique source locations¹⁶. This number is further split in half between native code source locations and dynamic source locations (for example, when Lua instrumentation is used).
- If there are recursive zones at any point in a zone stack, each unique zone source location should not appear more than 255 times.
- Profiling session cannot be longer than 1.6 days (2^{47} ns). This also includes on-demand sessions.
- No more than 4 billion (2^{32}) memory free events may be recorded.
- No more than 16 million (2^{24}) unique call stacks can be captured.

also: **callstack depth** limited to **about 60** stack frames

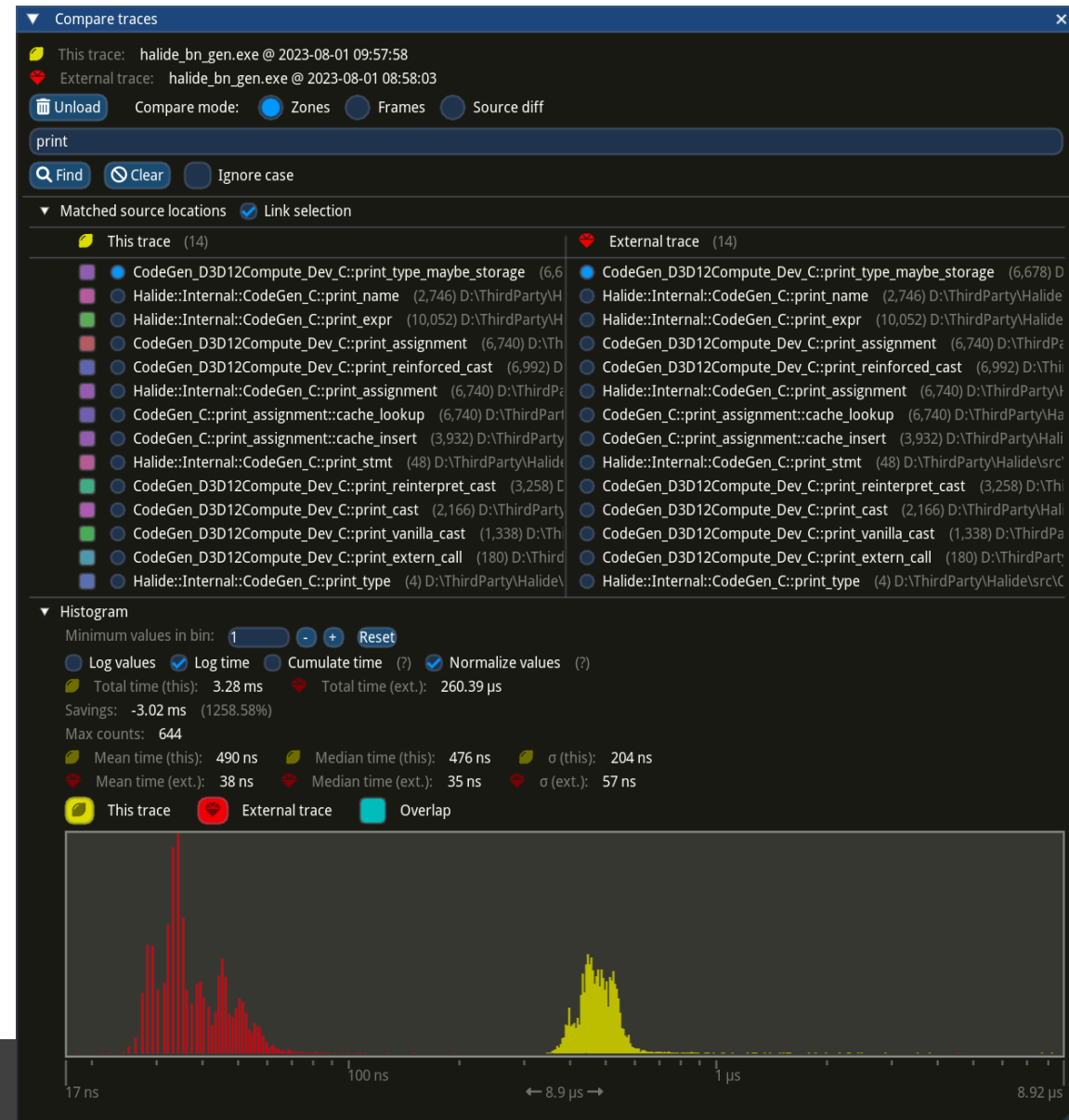
The following conditions also need to apply but don't trouble yourself with them too much. You would probably already know if you'd be breaking any.

- Only little-endian CPUs are supported.
- Virtual address space must be limited to 48 bits.
- Tracy server requires CPU which can handle misaligned memory accesses.

Study Cases

- [Photoshop] Win32 API Detours
- [Photoshop] Multi-threading opportunity
- [Photoshop] Eliminating redundant computations
- [Photoshop] Addressing lock contention
- [Halide] 100x faster HLSL code generation!
- [Halide] Yeah... `std::stringstream` is kinda slow...
- [Substance 3D Modeler] Improved parallel shader compilation
- [Substance 3D Stager] Eliminating GPU<->CPU copies with GPU API interop

Comparing Traces



A Taste of Tracy's Sampling Mode

TracyClient.cpp is all you need!

then launch your app with **admin privileges**
and connect it to the Tracy Profiler

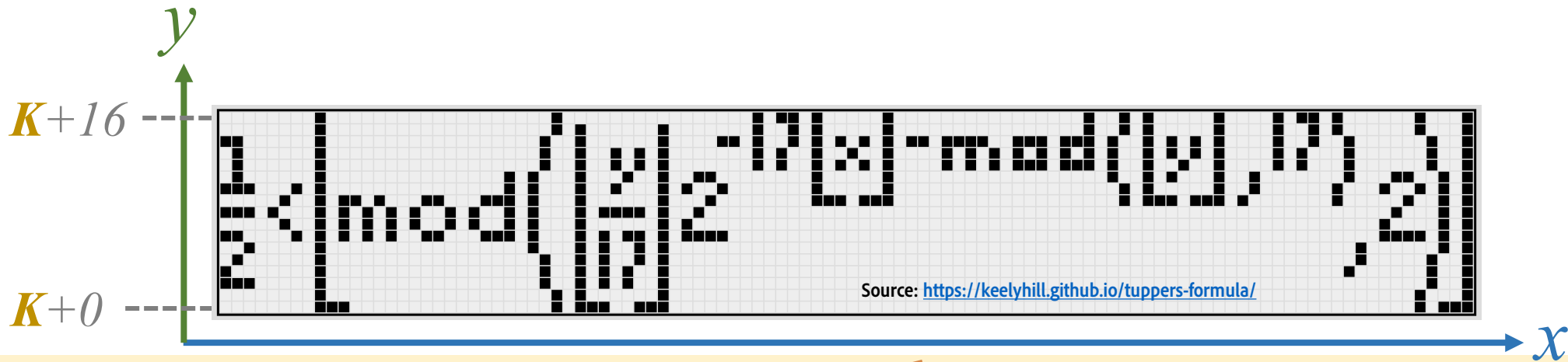
Manual instrumentation is **not necessary!**
(but you can have **both** coexist **together!**)



A Taste of Tracy's Sampling Mode

Tupper's Self-Referential Formula

$$\frac{1}{2} < \left\lfloor \text{mod} \left(\left\lfloor \frac{y}{17} \right\rfloor 2^{-17 \lfloor x \rfloor - \text{mod}(\lfloor y \rfloor, 17)}, 2 \right) \right\rfloor,$$



K =

48584506361897134235820959624942020445814005879832445494830930850619347047088099284506447698655243648499972470249151191104116057391774078569197543265718
554420572104457358836818298237541396343382251994521916512843483329051311703135502413758765239264874613394906870130562295813219481113685339535565290850
02387509285689269455597428154638651073004910672305893358605254409666735126534936364395712556569593681518433485760526694016125126695142155053955451915378
5457525756590740540157929001765967965480064427829131488548259914721248506352686630476300

544 digits

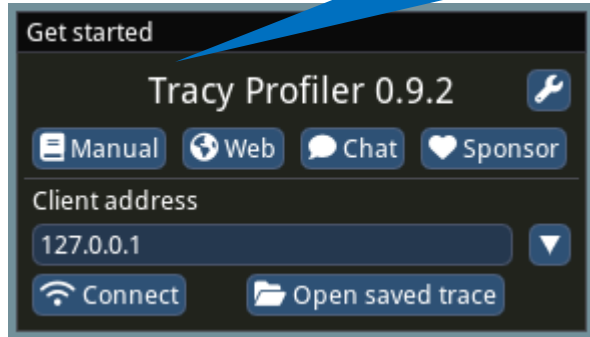
A New Speedrun Category!

Quake 2 Demo First Frame Any%

World Record: Marcos Slomp 1.32s

Help

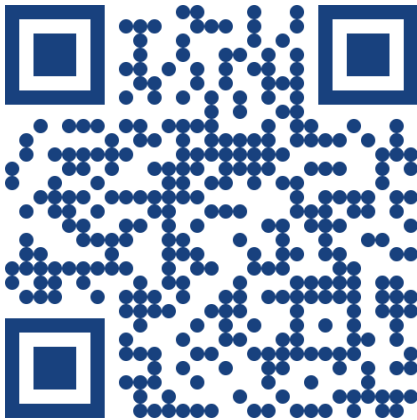
all the help you need is in the **profiler window** already!



RTTM: Read the Tracy Manual (it's **REALLY** good!)

tracy.pdf (grab it from <https://github.com/wolfpld/tracy/releases>)

Discord server



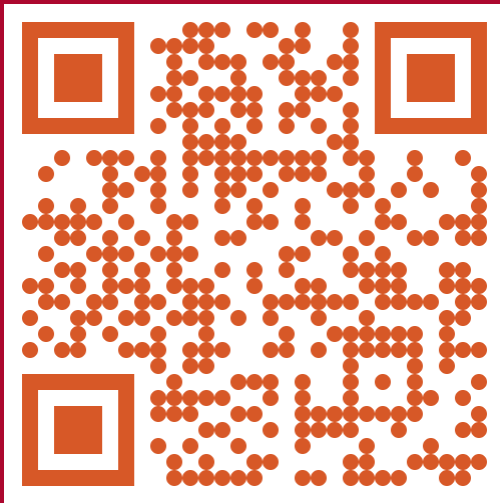
<https://discord.gg/pk78auc>

this->code

<https://github.com/slomp/cppcon/tracy>

C++ at Adobe!

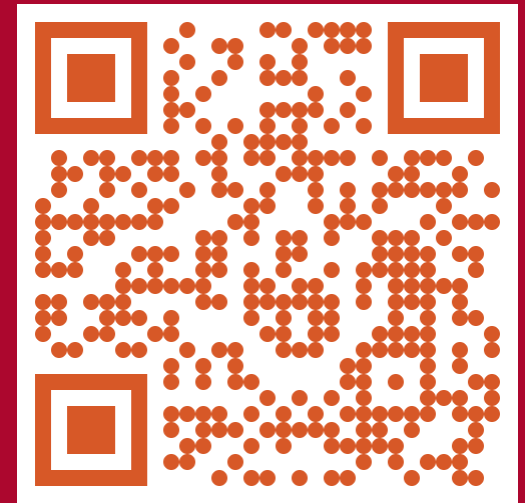
developer.adobe.com/cpp



- Careers
- Events
- Training
- Blog Posts



research.adobe.com



- Careers
- Internships
- Events
- Blog Posts

